

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: AN TOÀN ỨNG DỤNG WEB VÀ CƠ SỞ DỮ LIỆU
MÃ HỌC PHẦN: INT14105

ĐỀ TÀI: Tìm hiểu và triển khai chuỗi cân bằng tải các máy chủ web (web server load balancing cluster)

Sinh viên thực hiện:

B23DCAT040 Phan Thành Danh

B23DCAT326 Nghiêm Xuân Hoàng Tùng

B23DCAT329 Trịnh Thanh Tùng

Nhóm: 15

Lớp: 04

Giảng viên hướng dẫn: ThS Vũ Minh Mạnh

HỌC KỲ 1 NĂM HỌC 2026-2027

PHÂN CÔNG NHIỆM VỤ NHÓM THỰC HIỆN

TT	Công việc / Nhiệm vụ	SV thực hiện	Tỉ lệ phần trăm đóng góp
1	Tìm hiểu lý thuyết về cân bằng tải và các thuật toán phân phối	Nghiêm Xuân Hoàng Tùng	
2	Thiết kế và triển khai cụm ba máy chủ web với nginx load balancer	Phan Thành Danh	
3	Triển khai kho phiên dùng chung và lớp bảo mật tại cân bằng tải	Trịnh Thanh Tùng	
4	Thử nghiệm, thu số liệu phân phối tải, failover và đánh giá	Phan Thành Danh	

NHÓM THỰC HIỆN TỰ ĐÁNH GIÁ

TT	SV thực hiện	Thái độ tham gia	Mức hoàn thành CV	Kỹ năng giao tiếp	Kỹ năng hợp tác	Kỹ năng lãnh đạo
1	Phan Thành Danh					
2	Nghiêm Xuân Hoàng Tùng					
3	Trịnh Thanh Tùng					

MỤC LỤC

PHÂN CÔNG NHIỆM VỤ NHÓM THỰC HIỆN	2
NHÓM THỰC HIỆN TỰ ĐÁNH GIÁ	2
MỞ ĐẦU	10
CHƯƠNG 1. TỔNG QUAN VỀ CÂN BẰNG TẢI CÁC MÁY CHỦ WEB	11
1.1 Khái niệm	11
1.2 Cân bằng tải trong bối cảnh bảo mật ứng dụng web	11
1.2.1 Vị trí trong các lớp bảo mật	11
1.2.2 Quan hệ với các tấn công gây mất sẵn dùng	12
1.2.3 Bốn câu hỏi an toàn đặt ra cho mô hình	12
1.3 Phân tầng của bộ cân bằng tải	12
1.3.1 Cân bằng tải tầng 4	12
1.3.2 Cân bằng tải tầng 7	12
1.3.3 Cân bằng tải tầng 3 và cân bằng tải bằng DNS	12
1.4 Các thành phần của cụm máy chủ web cân bằng tải	13
1.5 Các thuật toán phân phối yêu cầu	14
1.5.1 Round robin	14
1.5.2 Weighted round robin	15
1.5.3 Least connection	15
1.5.4 Weighted least connection	16
1.5.5 IP hash	16
1.5.6 Hash theo URI và các thuật toán dựa trên nội dung	16
1.6 Kiểm tra sức khỏe node	17
1.6.1 Kiểm tra thụ động	17
1.6.2 Kiểm tra chủ động	17
1.6.3 Yêu cầu với đường dẫn kiểm tra sức khỏe	17
1.7 Duy trì phiên làm việc trong cụm	17
1.7.1 Vấn đề	17
1.7.2 Hai cách giải quyết	18
1.7.3 Hệ quả với bảo mật phiên	18
1.8 Các công cụ cân bằng tải phổ biến	18
1.9 Kết chương	19
CHƯƠNG 2. THIẾT KẾ VÀ TRIỂN KHAI CỤM MÁY CHỦ WEB CÂN BẰNG TẢI	20
2.1 Yêu cầu đặt ra	20
2.2 Kiến trúc tổng thể	20

2.2.1 Hai mạng và lý do tách.....	20
2.2.2 Quy hoạch địa chỉ.....	21
2.2.3 Vì sao phải có một container máy đo riêng.....	22
2.3 Tổ chức mã nguồn.....	22
2.4 Triển khai node web.....	23
2.4.1 Ảnh container của node.....	23
2.4.2 nginx trên node.....	23
2.4.3 Ứng dụng demo.....	24
2.5 Cấu hình bộ cân bằng tải.....	24
2.5.1 Khối upstream.....	24
2.5.2 Chuyển tiếp và phát hiện lỗi.....	25
2.5.3 Định dạng log.....	25
2.6 Kho phiên dùng chung.....	26
2.7 Lớp bảo mật của cụm.....	26
2.7.1 Kết thúc TLS và header phản hồi.....	26
2.7.2 Giới hạn tần suất.....	27
2.7.3 Cách ly và IP khách hàng.....	27
2.7.4 Đường kiểm tra sức khỏe và các đường dùng riêng cho phép đo.....	28
2.7.5 Đánh đổi của kết thúc TLS tập trung.....	28
2.8 Hướng loại bỏ điểm hỏng đơn lẻ còn lại.....	29
2.9 Kết chương.....	29
CHƯƠNG 3. THỬ NGHIỆM VÀ ĐÁNH GIÁ.....	30
3.1 Môi trường và cách tiến hành.....	30
3.1.1 Môi trường phần cứng và phần mềm.....	30
3.1.2 Cách tiến hành.....	30
3.1.3 Lệnh đo và cách đọc kết quả.....	31
3.2 Khả năng phân phối yêu cầu.....	33
3.2.1 Phép đo tuần tự.....	33
3.2.2 Phép đo đồng thời hai mươi lượt.....	34
3.2.3 Nhận xét.....	35
3.3 Khả năng đáp ứng khi một máy chủ gặp sự cố.....	36
3.3.1 Diễn biến.....	36
3.3.2 Bảng chứng trong log của bộ cân bằng tải.....	37
3.3.3 Vì sao không có request nào lỗi.....	38
3.4 Tính liên tục của phiên làm việc.....	39

3.5 Hiệu năng	40
3.5.1 Thông qua theo thuật toán và theo tầng phản hồi	41
3.5.2 Bậc thang chi phí giữa các tầng	43
3.5.3 Ba thuật toán có khác nhau về thông qua không.....	43
3.5.4 Hiệu năng khi mất một node	45
3.6 Kiểm tra cấu hình và một số thuộc tính an toàn.....	47
3.7 Nhận xét và hạn chế	49
3.8 Kết chương	50
KẾT LUẬN.....	51
TÀI LIỆU THAM KHẢO	54

DANH MỤC CÁC HÌNH VẼ

Hình 1 Hệ thống có bộ cân bằng tải đứng trước cụm máy chủ web	11
Hình 2 Các thành phần của một cụm máy chủ web cân bằng tải.....	13
Hình 3 Nguyên lý round robin	15
Hình 4 Nguyên lý least connection	16
Hình 5 Nguyên lý ip hash.....	16
Hình 6 Phiên đề trên từng node so với phiên đề ở kho dùng chung	18
Hình 7 Kiến trúc triển khai của cụm ba máy chủ web	20
Hình 8 Đường đi của một yêu cầu qua cụm.....	21
Hình 9 Kiểm tra cấu hình mạng: mạng backend khai báo internal nên không có đường ra ngoài .	21
Hình 10 Trạng thái cụm sau khi khởi động: bảy container đang chạy và chỉ lb01 có cổng công bố ra máy chủ	22
Hình 11 Một đáp ứng HTTPS đi qua cụm: header Server không kèm phiên bản, đủ bốn header an toàn, cookie phiên mang cả ba cờ	27
Hình 12 Gọi thẳng vào node từ mạng phía ngoài: curl không thiết lập được kết nối	28
Hình 13 Container khác trong cùng mạng backend gọi tới node bị trả về mã 403	28
Hình 14 Đường dẫn vào thư viện riêng của ứng dụng bị node chặn bằng mã 403	28
Hình 15 Danh sách file kết quả của lượt chạy được giữ lại làm nguồn cho mọi số liệu trong chương	31
Hình 16 Phép đo tuần tự với round robin.....	33
Hình 17 Phép đo tuần tự với least_conn, cho cùng chuỗi phân phối như round robin	34
Hình 18 Phép đo tuần tự với ip_hash, toàn bộ request đi về một node.....	34
Hình 19 Trái: số request trung bình mỗi node theo thuật toán, sai số là nửa khoảng tin cậy 95%. Phải: độ lệch lớn nhất giữa hai node đo trong cùng một lượt	35
Hình 20 Bảng phân phối hai mươi lượt in ra từ analyze_bench.py, đúng nguồn của các con số trong bảng trên.....	35
Hình 21 Mỗi chấm là một request trong vòng đo, tô theo node phục vụ. Đường đỏ là thời điểm tắt web02; đường xanh là request đầu tiên web02 phục vụ trở lại sau sự cố, trễ hơn lệnh bật node vài giây	37
Hình 22 Vòng đo failover: tắt web02 ở giây thứ 10, bật lại ở giây thứ 40, kèm bảng tổng hợp	37
Hình 23 Các dòng access log tương ứng các mẫu chậm, mỗi dòng có hai địa chỉ upstream và cặp mã 504 rồi 200.....	38
Hình 24 Phương án redis: tắt đúng node vừa phục vụ đăng nhập, mười lần gọi sau vẫn giữ phiên	40
Hình 25 Phương án file: cùng kịch bản, cả mười lần gọi bị trả về trang đăng nhập.....	40
Hình 26 Trái: thông qua theo tầng trên thang logarit, sai số là nửa khoảng tin cậy 95%. Phải: cùng phép đo ở tầng ứng dụng, nơi công việc thật của node xuất hiện	42

Hình 27 Bảng ma trận 18 ô in ra từ chương trình tổng hợp, đúng nguồn của các con số trong mục này	42
Hình 28 Đoạn in ra của cùng chương trình chứa mười hai phép kiểm định Welch và bậc thang chi phí ở mục 3.5.2.....	45
Hình 29 Bảng tổng hợp hai kích thước cụm in ra từ cùng chương trình sinh ra bảng trên, kèm dòng đếm node phục vụ	47
Hình 30 Bảng kiểm cấu hình mười chín hạng mục in ra từ sec_check.sh	49
Hình 31 Giới hạn tần suất chặn 194 trong 200 request gửi dồn dập vào đường thăm dò.....	49

DANH MỤC CÁC BẢNG BIỂU

Bảng 1. So sánh bộ cân bằng tải theo tầng làm việc	13
Bảng 2. So sánh các thuật toán phân phối yêu cầu	16
Bảng 3. So sánh các công cụ cân bằng tải thường dùng	19
Bảng 4. Bảng địa chỉ và cổng của các thành phần	22
Bảng 5. Cấu trúc thư mục của đề tài	22
Bảng 6. Các tham số đưa ra biến môi trường	25
Bảng 7. Môi trường thử nghiệm	30
Bảng 8. Ý nghĩa từng tham số của lệnh đo	31
Bảng 9. Ba endpoint dùng để tách chi phí theo tầng	32
Bảng 10. Phân phối 300 request gửi tuần tự	33
Bảng 11. Phân phối 600 request gửi đồng thời qua 20 kết nối, tổng hợp từ 20 lượt độc lập cho mỗi thuật toán	34
Bảng 12. Kết quả vòng đo khi tắt web02, trung bình 12 vòng lặp độc lập kèm nửa khoảng tin cậy 95%	36
Bảng 13. Giữ phiên sau khi tắt node đã phục vụ đăng nhập	39
Bảng 14. Thông qua trung bình của 20 lượt đo cho mỗi cặp thuật toán và endpoint. Cột CV là hệ số biến thiên, P50 và P99 tính trên từng request	41
Bảng 15. Chi phí cộng thêm của từng tầng, đo với least_conn	43
Bảng 16. Kiểm định Welch so với round robin trên cùng endpoint và cùng chế độ kết nối, mỗi bên 20 mẫu	44
Bảng 17. So sánh hai kích thước cụm, mỗi con số tổng hợp từ 20 lượt đo	45
Bảng 18. Kết quả kiểm tra cấu hình và thuộc tính an toàn của cụm, theo đúng thứ tự mà sec_check.sh in ra	47

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh	Thuật ngữ tiếng Việt
CSRF	Cross-Site Request Forgery	Giả mạo yêu cầu từ một site khác
CI	Confidence Interval	Khoảng tin cậy của giá trị trung bình
CV	Coefficient of Variation	Hệ số biến thiên, bằng độ lệch chuẩn chia trung bình
DNS	Domain Name System	Hệ thống tên miền
HSTS	HTTP Strict Transport Security	Chính sách buộc trình duyệt chỉ dùng HTTPS
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
HTTPS	HTTP Secure	HTTP chạy trên TLS
IP	Internet Protocol	Giao thức liên mạng
LVS	Linux Virtual Server	Dự án cân bằng tải tầng 4 của Linux
NAT	Network Address Translation	Phép chuyển đổi địa chỉ mạng
IETF	Internet Engineering Task Force	Tổ chức tiêu chuẩn hóa kỹ thuật Internet
OSI	Open Systems Interconnection	Mô hình tham chiếu kết nối mở
OWASP	Open Worldwide Application Security Project	Dự án mở về an toàn ứng dụng
PHP	Hypertext Preprocessor	Ngôn ngữ lập trình phía máy chủ
RST	Reset	Cờ TCP báo hủy kết nối đột ngột
RFC	Request for Comments	Văn bản chuẩn của IETF
TLS	Transport Layer Security	Giao thức bảo vệ kênh truyền
TCP	Transmission Control Protocol	Giao thức điều khiển truyền
UDP	User Datagram Protocol	Giao thức dữ liệu người dùng
URI	Uniform Resource Identifier	Bộ nhận dạng tài nguyên
WSL	Windows Subsystem for Linux	Hệ con Linux của Windows
VRRP	Virtual Router Redundancy Protocol	Giao thức dự phòng bộ định tuyến ảo

MỞ ĐẦU

Hệ thống thông tin của một cơ sở giáo dục chịu tải rất không đều. Trong vài chục phút đăng ký học phần, lượng người truy cập có thể gấp hàng chục ngày thường, và hệ thống chỉ cần chậm hoặc ngừng trong khoảng đó là ảnh hưởng trực tiếp tới việc học của hàng nghìn người. Với môn học về an toàn ứng dụng web, vấn đề này dẫn tới một yêu cầu cụ thể: làm thế nào để dịch vụ vẫn trả lời được khi có thành phần trong hệ thống gặp sự cố.

Ba yêu cầu cơ bản của an toàn thông tin là bí mật, toàn vẹn và sẵn dùng. Hai yêu cầu đầu đã được học phần xử lý kỹ ở các chương về tấn công và về bảo mật cơ sở dữ liệu. Yêu cầu thứ ba, tính sẵn dùng, được giáo trình nêu gọn ở mục về các yêu cầu bảo mật cơ sở dữ liệu, trong đó cân bằng tải máy chủ web cùng với nhân bản cơ sở dữ liệu là hai biện pháp bảo đảm hệ thống còn phục vụ được. Đề tài số 7, tìm hiểu và triển khai chuỗi cân bằng tải các máy chủ web, chọn đúng khoảng trống này để làm thực nghiệm.

Mục tiêu của đề tài là xây dựng một cụm gồm ba máy chủ web phục vụ cùng một website đứng sau một bộ cân bằng tải, rồi trả lời bằng số liệu ba câu hỏi mà đề bài nêu: yêu cầu có phân phối đều không, hệ thống còn đáp ứng được khi một máy chủ gặp sự cố không, và phiên làm việc của người dùng sống sót qua sự cố đó không. Kèm theo ba câu hỏi đó là một yêu cầu về phương pháp: mọi con số nêu trong báo cáo phải đọc lại được từ log thô do hệ thống tự sinh, không có số liệu nào chép tay.

Đối tượng nghiên cứu là cơ chế cân bằng tải ở tầng ứng dụng, các thuật toán phân phối yêu cầu, cách phát hiện node hỏng và cách giữ phiên làm việc trong cụm. Phạm vi giới hạn ở một cụm ba node trên một máy chủ, dùng nginx cho cả hai vai trò cân bằng tải và máy chủ web, MariaDB dùng chung và Redis làm kho phiên. Đề tài không triển khai cân bằng tải ở tầng bốn bằng LVS, không dựng cụm nhiều trung tâm dữ liệu, và không đánh giá khả năng chống tấn công từ chối dịch vụ quy mô lớn.

Phương pháp thực hiện gồm bốn bước. Nghiên cứu tài liệu để dựng khung lý thuyết về phân tầng, thuật toán và kiểm tra sức khỏe. Thiết kế và triển khai cụm bằng Docker Compose với hai mạng tách biệt. Xây dựng bộ script đo tự động, mỗi phép đo lưu cả bảng tổng hợp lẫn log thô. Cuối cùng là đối chiếu kết quả đo với khung lý thuyết ở Chương 1, trong đó có một phép đo đối chứng để chỉ rõ vì sao phải đưa phiên ra ngoài node.

Báo cáo gồm ba chương. Chương 1 trình bày khái niệm cân bằng tải máy chủ web, vị trí của nó trong yêu cầu bảo đảm tính sẵn dùng, phân tầng hoạt động, các thành phần của một cụm, các thuật toán phân phối, cơ chế kiểm tra sức khỏe và bài toán duy trì phiên. Chương 2 nêu yêu cầu đặt ra, thiết kế kiến trúc, cách triển khai node web, cấu hình bộ cân bằng tải, kho phiên dùng chung và các biện pháp bảo mật áp dụng cho cụm. Chương 3 trình bày môi trường, cách tiến hành và kết quả đo phân phối tải theo ba thuật toán, đo khả năng chịu lỗi khi tắt hẳn một node, đo tính liên tục của phiên, một ma trận 18 ô đo thông qua theo ba thuật toán và ba tầng phản hồi, cùng bảng kiểm cấu hình và thuộc tính an toàn gồm mười chín hạng mục.

CHƯƠNG 1. TỔNG QUAN VỀ CÂN BẰNG TẢI CÁC MÁY CHỦ WEB

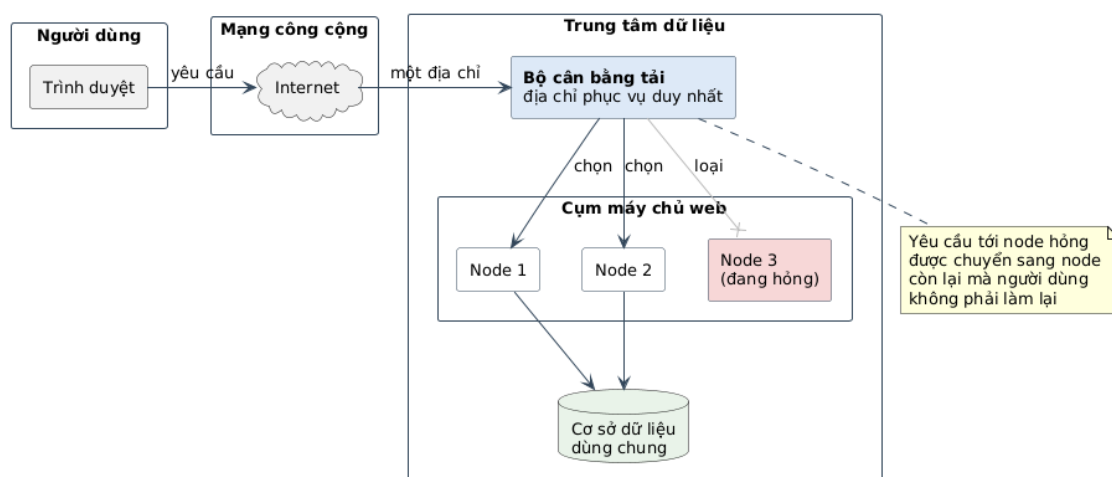
1.1 Khái niệm

Cân bằng tải (load balancing) là kỹ thuật phân chia khối lượng công việc của một hệ thống ra nhiều thành phần cùng đảm nhận, sao cho không thành phần nào phải chịu toàn bộ tải trong khi các thành phần khác còn nhàn rỗi, mô tả ở Hình 1. Khi áp dụng cho máy chủ web, cân bằng tải là việc phân phối các yêu cầu HTTP và HTTPS của người dùng tới nhiều máy chủ web cùng phục vụ một nội dung, thay vì dồn toàn bộ về một máy chủ duy nhất.

Một cụm máy chủ web (web server cluster) là tập hợp từ hai máy chủ web trở lên, cấu hình giống nhau, trả lời trên cùng một tên miền và cùng một nội dung. Bên ngoài, người dùng chỉ nhìn thấy một địa chỉ. Bên trong, yêu cầu được chia cho các thành viên của cụm.

Bộ cân bằng tải (load balancer) là phần mềm hoặc thiết bị đứng trước cụm máy chủ web, tiếp nhận yêu cầu từ người dùng, chọn ra một máy chủ trong cụm và chuyển yêu cầu đó tới máy chủ được chọn. Đáp ứng của máy chủ đi ngược trở lại người dùng thông qua bộ cân bằng tải, nên người dùng không biết và không cần biết máy chủ nào đã phục vụ mình.

Xét theo ba yêu cầu cơ bản của an toàn thông tin, cân bằng tải máy chủ web tác động trực tiếp vào tính sẵn dùng. Giáo trình của học phần xếp cân bằng tải cùng với nhân bản cơ sở dữ liệu thành hai biện pháp bảo đảm tính sẵn dùng, thể hiện ở Hình 5.12 của chương về an toàn cơ sở dữ liệu [1]. Đây là điểm khác biệt cần nhấn mạnh so với cách hiểu phổ biến coi cân bằng tải chỉ là giải pháp tăng tốc độ: tăng năng lực phục vụ là hệ quả, còn mục tiêu an toàn là giữ cho dịch vụ trả lời được khi một thành phần ngừng hoạt động.



Hình 1 Hệ thống có bộ cân bằng tải đứng trước cụm máy chủ web

1.2 Cân bằng tải trong bối cảnh bảo mật ứng dụng web

1.2.1 Vị trí trong các lớp bảo mật

Giáo trình chia việc bảo mật ứng dụng web thành các lớp, trong đó lớp bảo mật máy chủ là lớp nằm ngay dưới lớp ứng dụng [1]. Bộ cân bằng tải tạo ra một điểm tập trung để đặt các biện pháp của lớp này: chỉ một chỗ phải cấu hình TLS, chỉ một chỗ khai báo header

an toàn, chỉ một chỗ áp giới hạn tần suất. Ở mô hình một máy chủ đơn lẻ, các biện pháp đó nằm rải rác và dễ bị bỏ sót khi có máy chủ mới được thêm vào cụm.

1.2.2 Quan hệ với các tấn công gây mất sẵn dùng

Tấn công từ chối dịch vụ nhắm vào tính sẵn dùng có hai dạng. Dạng khai thác lỗ hổng làm sập tiến trình phục vụ chỉ cần một yêu cầu độc hại. Dạng chiếm hết tài nguyên cần lưu lượng lớn hơn khả năng đáp ứng. Cân bằng tải chỉ xử lý được một phần của dạng thứ hai, bằng cách mở rộng số node tiếp nhận yêu cầu và bằng cách loại node đã chết ra khỏi vòng phục vụ. Nó không xử lý được dạng khai thác lỗ hổng, cũng không xử lý được tấn công làm nghẽn đường truyền trước bộ cân bằng tải. Giới hạn này cần nói rõ để không trình bày cân bằng tải như một biện pháp chống từ chối dịch vụ hoàn chỉnh.

1.2.3 Bốn câu hỏi an toàn đặt ra cho mô hình

Đưa thêm một thành phần vào giữa người dùng và máy chủ làm xuất hiện các câu hỏi mà mô hình một máy chủ không có. Đoạn mã hóa từ bộ cân bằng tải về node có được bảo vệ không. Định danh phiên và cookie xử lý thế nào khi người dùng lần lượt gặp các node khác nhau. Nhật ký ở đâu phản ánh đúng địa chỉ của người gửi. Node có chấp nhận yêu cầu đi vòng qua bộ cân bằng tải hay không. Bốn câu hỏi này xuyên suốt đề tài và là nội dung của Chương 2 và Chương 3.

1.3 Phân tầng của bộ cân bằng tải

Bộ cân bằng tải được phân loại theo tầng của mô hình OSI mà nó đọc được thông tin để ra quyết định định tuyến.

1.3.1 Cân bằng tải tầng 4

Làm việc với địa chỉ IP và cổng TCP hoặc UDP. Bộ cân bằng tải nhận kết nối, chọn máy chủ đích theo bảng định tuyến hoặc theo thuật toán, rồi chuyển tiếp. Nội dung của giao thức ứng dụng không được giải mã, nên chi phí xử lý trên mỗi gói tin thấp và năng lực chuyển tiếp thường rất lớn.

Hệ quả là mọi quyết định chỉ dựa trên cặp địa chỉ và cổng. Hai yêu cầu tới /api và tới /images từ cùng một người dùng không thể đi tới hai cụm khác nhau.

1.3.2 Cân bằng tải tầng 7

Làm việc ở tầng ứng dụng, đọc được yêu cầu HTTP đã giải mã: phương thức, đường dẫn, tiêu đề, cookie. Nhờ đó bộ cân bằng tải định tuyến theo nội dung, chẳng hạn chuyển đường dẫn /api sang cụm ứng dụng, chuyển thư mục ảnh sang cụm phục vụ nội dung tĩnh, và chọn node dựa trên giá trị cookie.

Cân bằng tải tầng 7 còn thường đảm nhận giải mã TLS, gọi là kết thúc TLS. Việc này gom chứng chỉ về một chỗ và dồn chi phí tính toán mã hóa vào bộ cân bằng tải, đổi lại đoạn từ bộ cân bằng tải về node web có thể đi dưới dạng HTTP thuần nếu không được bảo vệ bằng một lớp mã hóa riêng.

1.3.3 Cân bằng tải tầng 3 và cân bằng tải bằng DNS

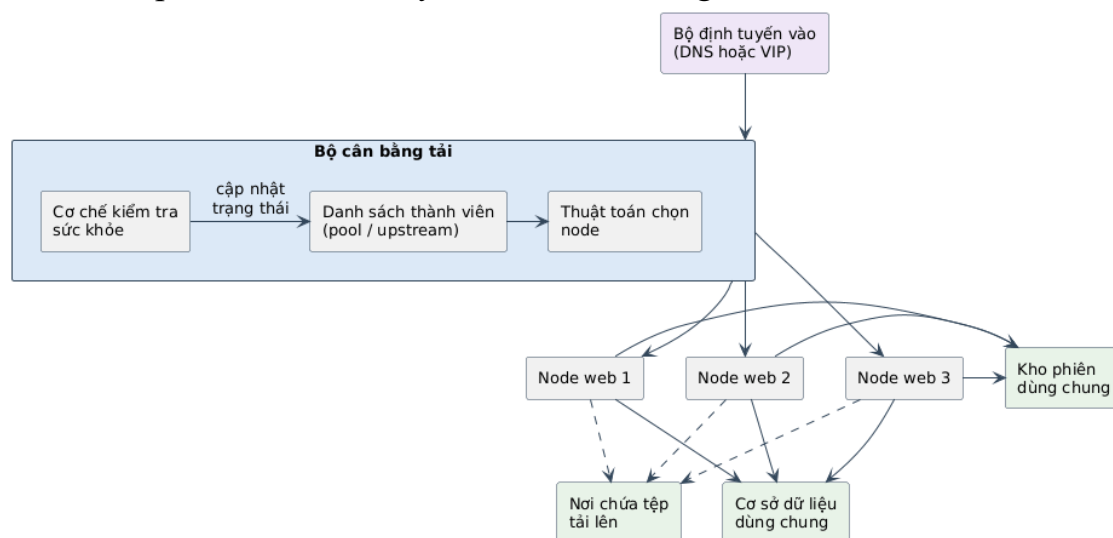
Ở tầng mạng, cân bằng tải dựa trên địa chỉ IP nguồn và IP đích, thường do thiết bị định tuyến đảm nhiệm, không liên quan tới nội dung web. Một cách tiếp cận khác phổ biến vì rẻ

là DNS round robin: tên miền trả về nhiều bản ghi A, trình duyệt tự phân tán kết nối. Hạn chế là bộ định tên miền không biết máy chủ nào còn sống, thời gian khuếch tán bản ghi khiến việc loại một máy chủ hỏng mất nhiều phút, và trình duyệt thường ghi nhớ kết quả nên một người dùng có thể dính mãi một máy chủ.

Bảng 1. So sánh bộ cân bằng tải theo tầng làm việc

Tiêu chí	Tầng 3	Tầng 4	Tầng 7
Thông tin dùng để quyết định	IP nguồn, IP đích	IP và cổng TCP/UDP	Nội dung HTTP: đường dẫn, tiêu đề, cookie
Có giải mã TLS tại bộ cân bằng tải	Không	Không bắt buộc	Có
Định tuyến theo đường dẫn URI	Không	Không	Có
Duy trì phiên theo cookie	Không	Không	Có
Chi phí xử lý mỗi yêu cầu	Thấp	Thấp	Trung bình đến cao
Phù hợp với ứng dụng web động	Kém	Tốt	Tốt nhất

1.4 Các thành phần của cụm máy chủ web cân bằng tải



Hình 2 Các thành phần của một cụm máy chủ web cân bằng tải

Bộ cân bằng tải giữ danh sách thành viên của cụm, còn gọi là pool hoặc upstream. Mỗi thành viên có địa chỉ, cổng, trọng số và trạng thái.

Cơ chế kiểm tra sức khỏe xác định thành viên nào đang đáp ứng được. Có hai cách. Kiểm tra thụ động chỉ nhìn kết quả của các yêu cầu thật: sau một số lần lỗi liên tiếp, thành viên bị đánh dấu không đạt và bị loại trong một khoảng thời gian, hết thời gian thì được cho vào lại. Kiểm tra chủ động gửi thêm các yêu cầu thăm dò định kỳ tới một đường dẫn dành riêng, nên phát hiện máy chủ chết trước khi người dùng thật gặp lỗi.

Kho lưu phiên dùng chung giữ dữ liệu phiên của mọi node tại một chỗ, để bất kỳ node nào cũng đọc được phiên của bất kỳ người dùng nào.

Lưu trữ dùng chung và hệ quản trị cơ sở dữ liệu dùng chung bảo đảm mọi node đọc và ghi trên cùng một tập dữ liệu. Nếu mỗi node có một cơ sở dữ liệu riêng thì hai người dùng gặp hai node khác nhau sẽ thấy hai nội dung khác nhau.

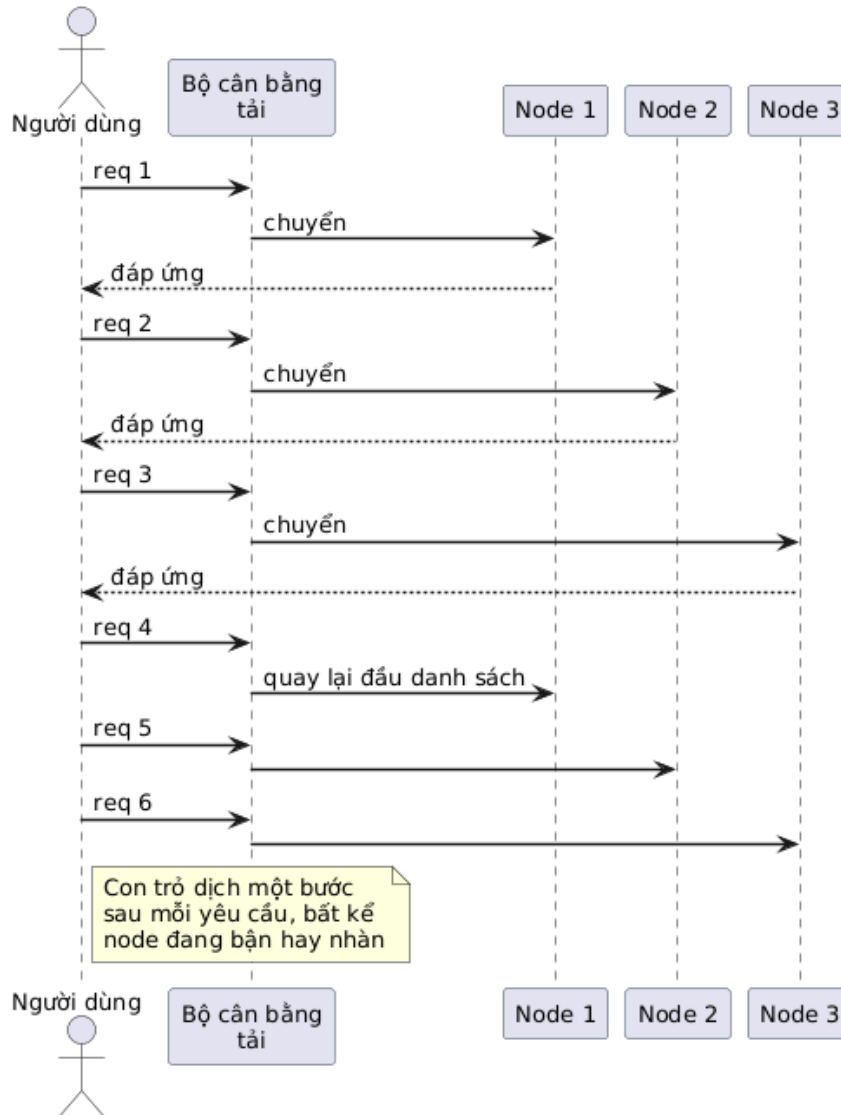
Cụm cũng cần một cơ chế để nội dung do người dùng tải lên xuất hiện trên mọi node. Các thành phần của cụm được vẽ ở Hình 2. Với hệ thống nhỏ, dữ liệu tải lên nên đưa vào cơ sở dữ liệu hoặc một vùng lưu trữ đối tượng, không để trong thư mục cục bộ của node.

1.5 Các thuật toán phân phối yêu cầu

1.5.1 Round robin

Duyệt qua danh sách thành viên theo thứ tự vòng. Yêu cầu thứ nhất tới node thứ nhất, yêu cầu thứ hai tới node thứ hai, hết danh sách thì quay lại. Thuật toán không xét tới việc node nào đang bận hay nhanh chậm, chỉ cần danh sách còn thành viên là còn phân đều theo số lượng, mô tả ở Hình 3.

Round robin: sáu yêu cầu đi qua ba node theo thứ tự vòng



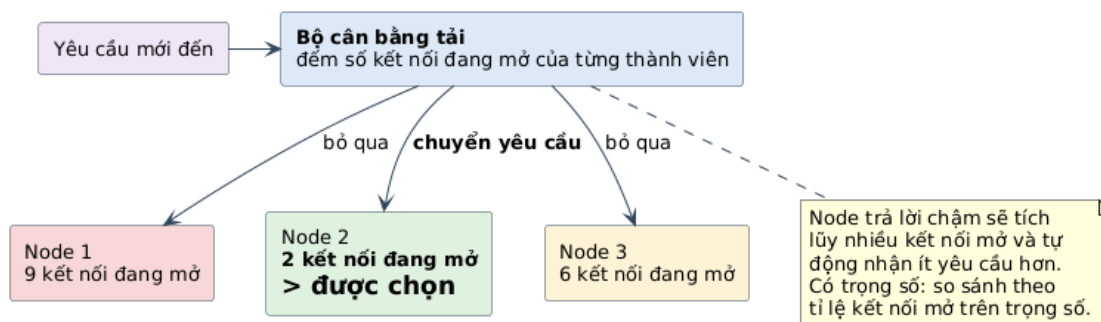
Hình 3 Nguyên lý round robin

1.5.2 Weighted round robin

Mỗi thành viên gán một trọng số là số nguyên. Trong một vòng, thành viên có trọng số lớn được chọn nhiều lần hơn theo đúng tỉ lệ trọng số. Dùng khi các máy chủ trong cụm có cấu hình phần cứng khác nhau. Round robin thuần là trường hợp đặc biệt khi mọi trọng số bằng nhau, mô tả ở Hình 3.

1.5.3 Least connection

Chọn thành viên đang có ít kết nối đang xử lý nhất, có tính trọng số. Khác với round robin, thuật toán này phản ánh tải thực tế tại thời điểm quyết định: một node trả lời chậm sẽ tích lũy nhiều kết nối mở và tự động nhận ít yêu cầu mới hơn. Ở chiều ngược lại, khi mọi node đồng chất và các request nhanh như nhau, số kết nối đang mở tại thời điểm chọn thường bằng nhau ở mọi node nên kết quả sát với round robin; khác biệt giữa hai thuật toán chỉ xuất hiện khi có node chậm hoặc khi nhiều request chồng lấn lên nhau, so sánh ở Hình 4 và mục 3.2.3.



Hình 4 Nguyên lý least connection

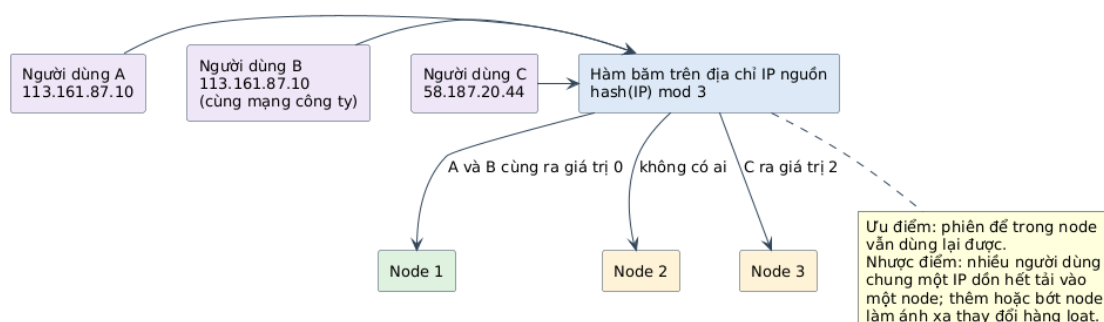
1.5.4 Weighted least connection

Kết hợp hai ý trên: chọn tỉ lệ giữa số kết nối đang mở và trọng số, rồi lấy giá trị nhỏ nhất.

1.5.5 IP hash

Tính hàm băm trên địa chỉ IP của người gửi, rồi ánh xạ kết quả vào danh sách thành viên. Cùng một địa chỉ IP luôn ra cùng một node, nên phiên để ở node vẫn được sử dụng lại.

Hai hạn chế. Người dùng trong một mạng công ty đi ra chung một địa chỉ sẽ đổ hết tải lên một node. Khi danh sách thành viên thay đổi, ánh xạ băm thay đổi theo và phần lớn người dùng bị chuyển sang node khác, mô tả ở Hình 5.



Hình 5 Nguyên lý ip hash

1.5.6 Hash theo URI và các thuật toán dựa trên nội dung

Băm đường dẫn yêu cầu để mọi yêu cầu tới cùng một tài nguyên đi về cùng một node, hoặc dùng một cookie riêng do bộ cân bằng tải gán để giữ người dùng ở lại node đã chọn. Nhóm này thuộc khả năng chỉ có ở cân bằng tải tầng 7.

Bảng 2. So sánh các thuật toán phân phối yêu cầu

Thuật toán	Căn cứ quyết định	Đều theo số request	Có tính tới tốc độ node	Giữ được phiên tại node	Nhảy khi thêm bớt node
Round robin	Thứ tự vòng	Có	Không	Không	Ít
Weighted round robin	Thứ tự vòng và trọng số	Theo trọng số	Một phần, qua trọng số	Không	Ít

Thuật toán	Căn cứ quyết định	Điều theo số request	Có tính tới tốc độ node	Giữ được phiên tại node	Nhạy khi thêm bớt node
Least connection	Số kết nối đang mở	Không tuyệt đối	Có	Không	Ít
Weighted least connection	Kết nối đang mở và trọng số	Không tuyệt đối	Có	Không	Ít
IP hash	Địa chỉ IP người gửi	Không	Không	Có	Nhiều
Hash theo URI	Đường dẫn yêu cầu	Không	Không	Một phần	Nhiều

1.6 Kiểm tra sức khỏe node

1.6.1 Kiểm tra thụ động

Trong nginx mã nguồn mở, cấu hình gồm hai tham số đi cùng nhau. `max_fails` là số lần yêu cầu thất bại liên tiếp cần thiết để đánh dấu một thành viên không đạt. `fail_timeout` là khoảng thời gian thành viên bị loại, đồng thời là cửa sổ đếm lỗi. Sau khi hết hạn, nginx gửi thử một yêu cầu; nếu thành công, thành viên trở lại danh sách.

Các lỗi được tính gồm kết nối bị từ chối, hết thời gian kết nối, hết thời gian đọc, và một số mã trạng thái nếu được khai báo trong chỉ thị `proxy_next_upstream`.

1.6.2 Kiểm tra chủ động

Kiểm tra chủ động cần một yêu cầu thăm dò riêng, gửi định kỳ tới một đường dẫn như `/healthz`, và coi là đạt khi nhận đúng mã trạng thái mong đợi. HAProxy hỗ trợ sẵn qua option `httpchk`. Nginx mã nguồn mở không có kiểm tra chủ động trong khối `upstream`; muốn có phải dùng nginx Plus hoặc một tiến trình ngoài ghi kết quả vào file mà nginx đọc lại.

Sự khác biệt này quyết định hình dạng của thời gian gián đoạn đo được ở Chương 3. Với kiểm tra thụ động, yêu cầu đầu tiên sau khi node chết chắc chắn thất bại hoặc phải chờ nginx chuyển sang node kế tiếp, vì nginx chỉ biết node hỏng sau khi thử phục vụ người dùng thật.

1.6.3 Yêu cầu với đường dẫn kiểm tra sức khỏe

Đường dẫn kiểm tra phải trả lời nhanh, không tạo phiên, không ghi cơ sở dữ liệu và không tiết lộ thông tin hệ thống. Một trang hiển thị phiên bản PHP hoặc đường dẫn tuyệt đối sẽ biến cơ chế giám sát thành kênh cung cấp thông tin cho người tấn công.

1.7 Duy trì phiên làm việc trong cụm

1.7.1 Vấn đề

Ứng dụng web là ứng dụng có trạng thái: sau khi đăng nhập, máy chủ giữ thông tin người dùng và gán nó với một định danh phiên gửi trong cookie. Giáo trình trình bày các yêu cầu với định danh phiên tại mục bảo mật phiên làm việc, trong đó có việc khởi tạo lại định danh sau khi người dùng đổi quyền [1].

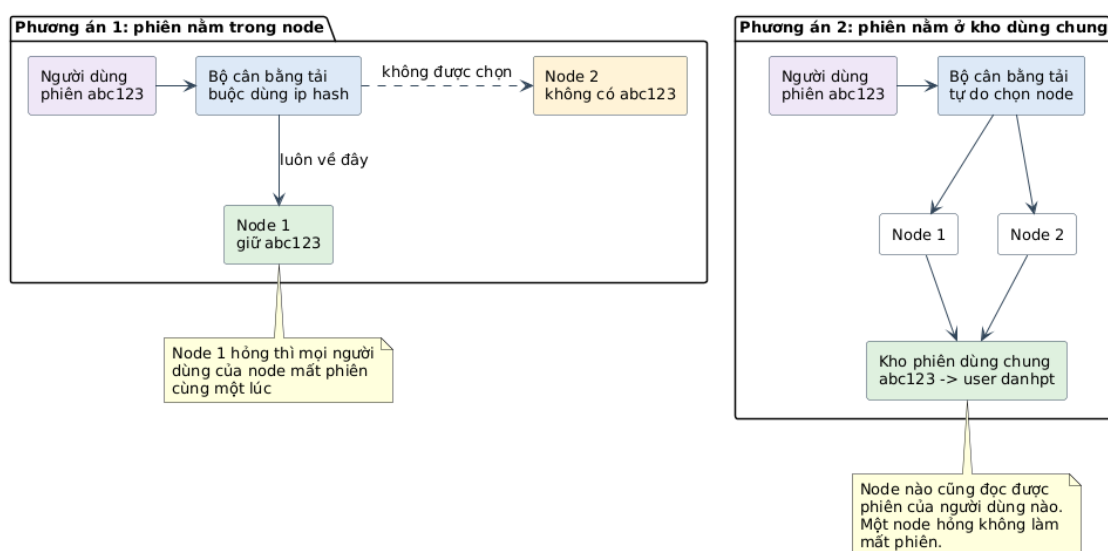
Trong cụm máy chủ web, vấn đề phát sinh vì phiên được tạo ra trên một node cụ thể. Nếu dữ liệu phiên chỉ nằm trong bộ nhớ hoặc trên đĩa của node đó, yêu cầu tiếp theo của cùng người dùng mà rơi vào node khác sẽ không tìm thấy phiên, và người dùng bị coi là chưa đăng nhập.

1.7.2 Hai cách giải quyết

Cách thứ nhất là giữ người dùng luôn quay về cùng một node, dùng ip hash, dùng cookie của bộ cân bằng tải, hoặc dùng phiên gắn với kết nối. Cách này không cần thay đổi ứng dụng nhưng làm giảm chất lượng cân bằng tải, và khi node bị giữ phiên ngừng hoạt động thì toàn bộ người dùng của node đó mất phiên cùng một lúc.

Cách thứ hai là đưa dữ liệu phiên ra ngoài node, vào một kho lưu trữ dùng chung như Redis hoặc vào chính cơ sở dữ liệu. Mọi node đọc được mọi phiên, nên bộ cân bằng tải tự do chọn node theo thuật toán nào cũng được, và một node chết không làm mất phiên.

Đề tài chọn cách thứ hai và dùng cách thứ nhất làm đối chứng trong phép đo ở Chương 3, so sánh ở Hình 6.



Hình 6 Phiên để trên từng node so với phiên để ở kho dùng chung

1.7.3 Hệ quả với bảo mật phiên

Khi phiên nằm ở kho dùng chung, một số biện pháp bảo vệ vẫn phải đặt đúng chỗ. Cookie phiên phải có cờ Secure để chỉ đi qua HTTPS, cờ HttpOnly để mã lệnh chèn vào trình duyệt không đọc được, và thuộc tính SameSite để chặn một phần tấn công giả mạo yêu cầu khác nguồn. Việc khởi tạo lại định danh phiên ngay sau khi đăng nhập thành công vẫn phải thực hiện, vì kho lưu trữ dùng chung không tự chống được tấn công cố định phiên.

Kho lưu phiên trở thành một điểm chứa dữ liệu nhạy cảm và phải nằm trong mạng nội bộ, không có cổng công bố ra ngoài. Ai lấy được định danh phiên hợp lệ thì còn nguy hiểm hơn trong mô hình một máy chủ, vì người đó có thể dùng phiên trên bất kỳ node nào.

1.8 Các công cụ cân bằng tải phổ biến

Bảng 3. So sánh các công cụ cân bằng tải thường dùng

Công cụ	Tầng làm việc	Kiểm tra sức khỏe chủ động	Duy trì phiên	Ghi chú
LVS	4	Có, qua daemon ngoài	Không	Chuyển tiếp gói tin ở tầng mạng, hiệu năng rất cao
HAProxy	4 và 7	Có, option httpchk	Có, qua cookie	Trang thống kê trực tiếp hiển thị trạng thái node
Nginx mã nguồn mở	4 và 7	Không, chỉ thụ động	Có, ip hash và sticky của bản thương mại	Phổ biến, tài liệu nhiều, tích hợp làm máy chủ web
keepalived	3 và 4	Có, qua vrrp_script	Không	Thường ghép với LVS hoặc HAProxy để chống điểm hỏng đơn lẻ
Cân bằng tải của nhà cung cấp đám mây	4 và 7	Có	Có	Cấu hình theo giao diện, chi phí theo lưu lượng

Đề tài chọn nginx cho cả vai trò bộ cân bằng tải và máy chủ web ở mỗi node. Lý do là nginx phổ biến trong các hệ thống web, tài liệu cấu hình ngắn gọn và dễ trình bày trong báo cáo, và việc dùng cùng một phần mềm ở hai vai trò giúp tách rõ khác biệt giữa cấu hình định tuyến và cấu hình phục vụ nội dung. Điểm yếu của lựa chọn này là nginx mã nguồn mở không có kiểm tra sức khỏe chủ động, nên thời gian phát hiện node hỏng phụ thuộc vào yêu cầu của người dùng thật; đây là hạn chế được đo và bàn luận ở Chương 3.

1.9 Kết chương

Chương này trình bày cân bằng tải máy chủ web như một biện pháp bảo đảm tính sẵn dùng trong mô hình an toàn ứng dụng web, xác định vị trí của bộ cân bằng tải trong kiến trúc nhiều lớp, phân biệt cân bằng tải ở tầng 4 và tầng 7, liệt kê các thành phần của một cụm máy chủ web gồm pool thành viên, cơ chế kiểm tra sức khỏe, kho phiên dùng chung và cơ sở dữ liệu dùng chung, so sánh các thuật toán phân phối yêu cầu và các công cụ thường dùng.

Bốn vấn đề được rút ra để làm mục tiêu cho phân triển khai. Thứ nhất, bộ cân bằng tải phải phân phối được yêu cầu đều trên các node theo một thuật toán chọn được. Thứ hai, khi một node ngừng trả lời, cụm phải tiếp tục phục vụ mà không cần can thiệp thủ công. Thứ ba, phiên làm việc của người dùng phải sống được qua sự cố của node, nên dữ liệu phiên không được nằm trong node. Thứ tư, node web không được phép tiếp nhận yêu cầu từ ngoài và không được tiết lộ thông tin qua đường kiểm tra sức khỏe. Chương 2 triển khai cụm đáp ứng bốn yêu cầu này, Chương 3 đo lại bằng số liệu.

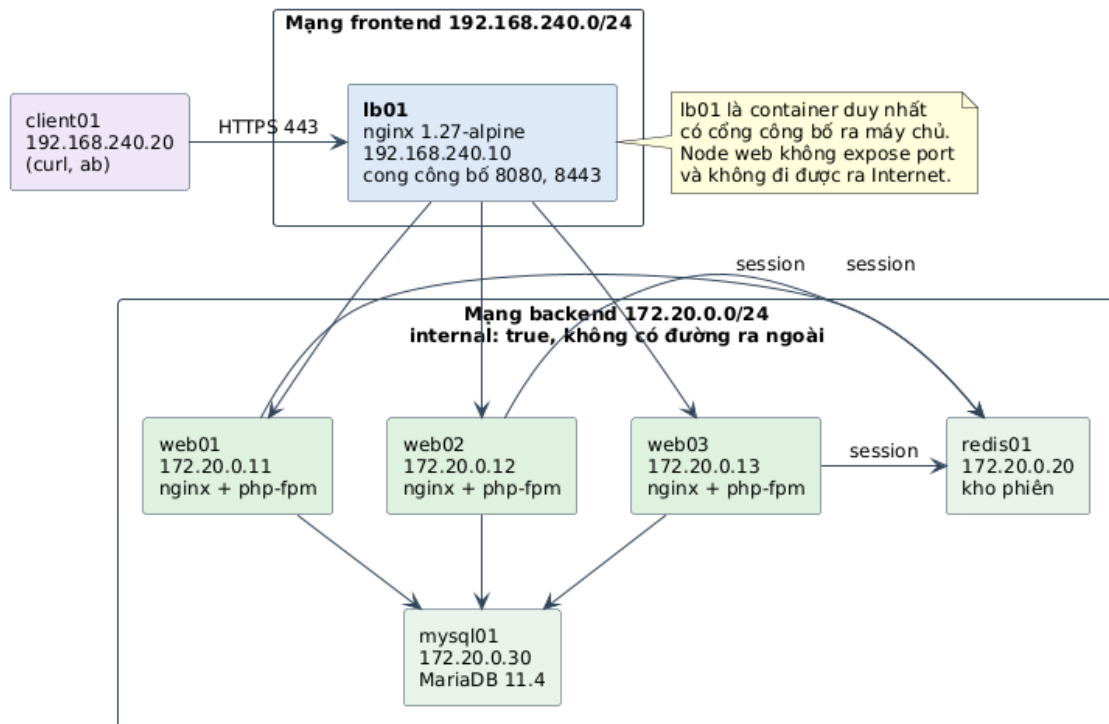
CHƯƠNG 2. THIẾT KẾ VÀ TRIỂN KHAI CỤM MÁY CHỦ WEB CÂN BẰNG TẢI

2.1 Yêu cầu đặt ra

Đề bài yêu cầu xây dựng một cụm gồm hai đến ba máy chủ web phục vụ cùng một website cùng một thành phần cân bằng tải, sau đó kiểm tra phân phối request và khả năng đáp ứng khi một server gặp sự cố. Từ yêu cầu đó và từ bốn vấn đề rút ra ở Kết chương 1, đề tài đặt ra sáu yêu cầu cụ thể cho phần triển khai.

Thứ nhất, cụm có ba node web độc lập, cùng một bản ảnh, cùng nội dung, và bộ cân bằng tải phải chọn được node theo thuật toán khai báo. Thứ hai, khi tắt hẳn một node, dịch vụ vẫn trả lời mà người dùng không phải làm lại thao tác. Thứ ba, phiên đăng nhập không nằm trong node, nên node chết không làm mất phiên. Thứ tư, dữ liệu người dùng đọc từ một cơ sở dữ liệu dùng chung, bảo đảm mọi node trả về cùng nội dung. Thứ năm, node web không được phép tiếp nhận kết nối từ ngoài và không được tiết lộ thông tin hệ thống. Thứ sáu, mọi phép đo ở Chương 3 phải chạy lại được bằng một lệnh và lưu được log thô.

2.2 Kiến trúc tổng thể



Hình 7 Kiến trúc triển khai của cụm ba máy chủ web

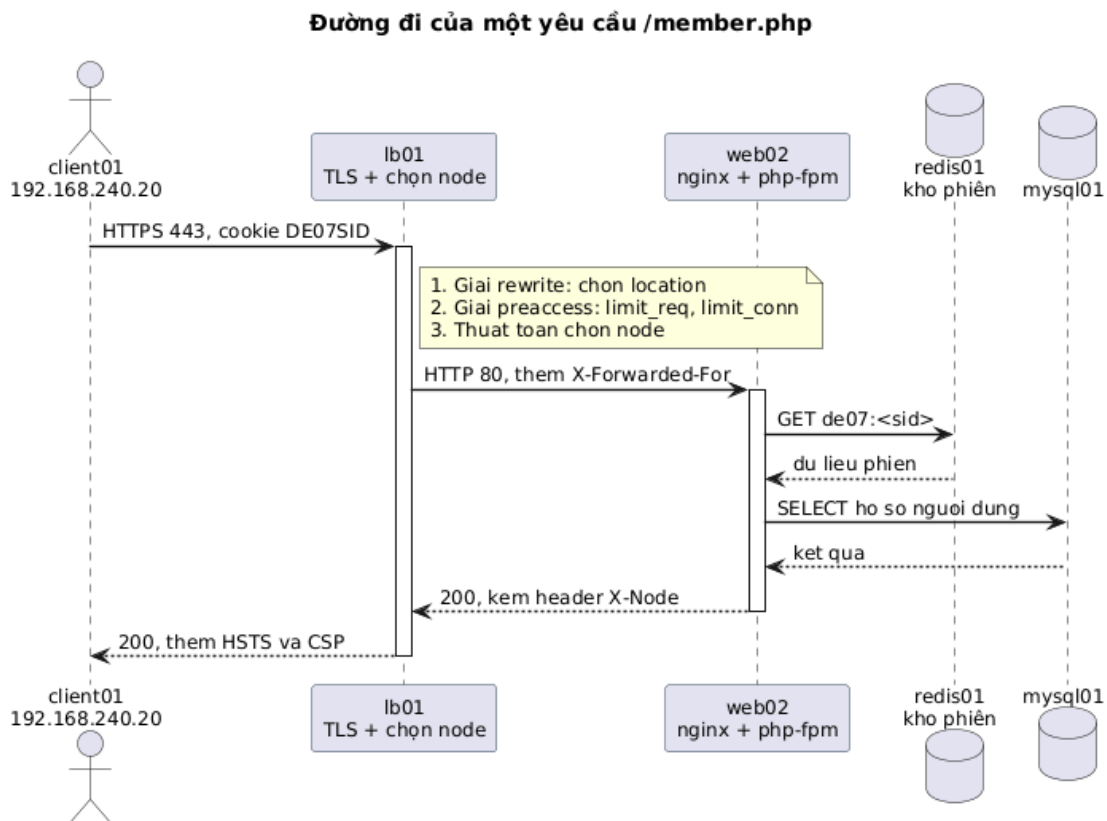
Toàn bộ hệ thống chạy trên một máy vật lý, tổ chức bằng Docker Compose. Mỗi thành phần là một container thật, nghe trên cổng thật và nói chuyện với nhau qua hai mạng ảo riêng biệt. Lưu thông giữa các thành phần là HTTP và TCP thật, không có lớp mô phỏng.

Việc gom nhiều container vào một máy chỉ thay đổi chỗ đặt thiết bị, không thay đổi đường đi của gói tin: yêu cầu vẫn đi từ máy khách tới bộ cân bằng tải, bộ cân bằng tải vẫn mở một kết nối TCP mới tới node, và node vẫn tự đọc cơ sở dữ liệu.

2.2.1 Hai mạng và lý do tách

Mạng frontend (192.168.240.0/24) nối máy khách với bộ cân bằng tải. Mạng backend (172.20.0.0/24) nối bộ cân bằng tải với ba node, kho phiên và cơ sở dữ liệu. Mạng backend khai báo `internal: true`, nghĩa là Docker không cấp cho nó đường ra bên ngoài và không định tuyến nó đi đâu cả.

Hệ quả kiểm chứng được bằng lệnh: gọi thẳng một node từ máy khách thì không nối được, gọi từ node này sang node khác trong cùng mạng thì vẫn bình thường. Đây là biện pháp cách ly chính của đề tài, đặt ở tầng mạng thay vì dựa vào cấu hình trên node.



Hình 8 Đường đi của một yêu cầu qua cụm

```

lb-lab-02 mạng backend là internal
=== C:/Users/Administrator/Documents/Qoder/2026-09-29/8b86a2bd/de07-web-lb-cluster ===
$ docker network inspect de07-web-lb-cluster_backend | grep -Ei "Name|Internal|Subnet"

    "Name": "de07-web-lb-cluster_backend",
    "Subnet": "172.20.0.0/24",
    "Internal": true,
    "Name": "de07-web02",
    "Name": "de07-web01",
    "Name": "de07-redis01",
    "Name": "de07-web03",
    "Name": "de07-mysql01",
    "Name": "de07-lb01",
    "Subnets": {
exit code: 0

```

Hình 9 Kiểm tra cấu hình mạng: mạng backend khai báo `internal` nên không có đường ra ngoài

2.2.2 Quy hoạch địa chỉ

Bảng 4. Bảng địa chỉ và cổng của các thành phần

Thành phần	Container	Địa chỉ	Vai trò	Cổng công bố ra máy
Bộ cân bằng tải	lb01	192.168.240.10 và 172.20.0.10	nginx 1.27, TLS, chọn node	8080, 8443
Node web 1	web01	172.20.0.11	nginx và php-fpm 8.3	không
Node web 2	web02	172.20.0.12	nginx và php-fpm 8.3	không
Node web 3	web03	172.20.0.13	nginx và php-fpm 8.3	không
Kho phiên	redis01	172.20.0.20	Redis 7, lưu session	không
Cơ sở dữ liệu	mysql01	172.20.0.30	MariaDB 11.4	không
Máy đo	client01	192.168.240.20	curl, ab	không

Chỉ lb01 có cổng công bố ra máy chủ. Máy khách trong mạng nội bộ gọi tới 192.168.240.10, còn khi cần xem bằng trình duyệt thì gọi qua cổng 8443 của máy chủ.

```

lb-lab-01 bay container, chỉ lb01 có cổng ngoài
$ bash scripts/lab.sh ps
de07-client01    Up 11 hours
de07-lb01        Up 5 minutes    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp, 0.0.0.0:8443->443/tcp,
[::]:8443->443/tcp
de07-mysql01     Up About an hour (healthy)    3306/tcp
de07-redis01     Up About an hour (healthy)    6379/tcp
de07-web01       Up 28 minutes   80/tcp
de07-web02       Up 4 minutes    80/tcp
de07-web03       Up 28 minutes   80/tcp
exit code: 0

```

Hình 10 Trạng thái cụm sau khi khởi động: bảy container đang chạy và chỉ lb01 có cổng công bố ra máy chủ

2.2.3 Vì sao phải có một container máy đo riêng

Nếu phát yêu cầu từ Windows thì Docker Desktop sẽ dịch địa chỉ nguồn qua NAT, mọi request đều mang cùng một địa chỉ và không phân biệt được. Hai hệ quả: phép thử ip hash cho kết quả vô nghĩa, và chuỗi IP trong log không còn là bằng chứng. Container client01 nằm cùng mạng frontend với bộ cân bằng tải nên địa chỉ nguồn là thật, giữ được ý nghĩa cho cả hai phép đo.

2.3 Tổ chức mã nguồn

Cây thư mục của đề tài như sau.

Bảng 5. Cấu trúc thư mục của đề tài

Đường dẫn	Nội dung
docker-compose.yml	Khai báo bảy service, hai mạng, một volume
nginx/lb/nginx.conf	Cấu hình gốc của bộ cân bằng tải, vùng giới hạn tần suất, định dạng log

Đường dẫn	Nội dung
nginx/lb/templates/default.conf.template	Khởi upstream và hai server block, điền tham số bằng biến môi trường
web/Dockerfile	Ảnh node web: Alpine 3.20, nginx, php-fpm 8.3, phpredis
web/nginx.conf	Cấu hình nginx trên từng node
web/entrypoint.sh	Khởi động php-fpm và nginx, sinh cấu hình phiên lúc chạy
web/app	Ứng dụng PHP demo
mysql/init/01_schema.sql	Bảng users, news, login_audit và dữ liệu mẫu
scripts	Sáu script đo và điều khiển
results	Log thô của mọi phép đo

2.4 Triển khai node web

2.4.1 Ảnh container của node

Ba node dùng chung một ảnh, chỉ khác nhau biến môi trường `NODE_NAME`. Ảnh dựng trên Alpine 3.20 với hai gói nginx và php83-fpm đóng gói sẵn của phân phối, nên không phải biên dịch gì.

Một chi tiết bắt buộc phải sửa: php-fpm đặt `clear_env = yes` theo mặc định, nghĩa là nó xóa toàn bộ biến môi trường của container trước khi chạy tiến trình con. Nếu không đổi thành `clear_env = no`, ứng dụng gọi `getenv('DB_HOST')` sẽ nhận giá trị rỗng và không nối được cơ sở dữ liệu, dù trong shell của container biến đó vẫn tồn tại. Đây là lỗi chỉ lộ ra khi chạy thật.

2.4.2 nginx trên node

Node nhận yêu cầu trên cổng 80, chuyển tệp tin tĩnh trực tiếp và đưa tệp .php sang php-fpm ở 127.0.0.1:9000. Đoạn cấu hình đáng chú ý:

```
allow 172.20.0.10;      # lb01
allow 127.0.0.1;       # kiểm tra nơi bo trong container
deny all;

add_header X-Node "@NODE_NAME@" always;
fastcgi_hide_header X-Powered-By;

map $http_x_real_ip $client_addr {
    default $http_x_real_ip;
    ""      $remote_addr;
}

location ~ /\.php$ {
    fastcgi_param REMOTE_ADDR $client_addr;
    ...
}
```

Ba dòng `allow/deny` buộc node chỉ phục vụ đúng bộ cân bằng tải. Cách viết tự nhiên là `set_real_ip_from` kết hợp `real_ip_header X-Forwarded-For` để `$remote_addr` luôn là IP khách hàng, nhưng làm vậy thì các quy tắc chặn lại lấy đúng cái đích cần chặn làm điều

kiện: module `realip` chạy ở giai đoạn `preaccess` trước module `access`, nên khi `deny` được xét thì `$remote_addr` đã bị ghi đè bằng địa chỉ đọc từ tiêu đề, tức là từ dữ liệu người gọi gửi lên. Node vì thế không dùng `realip` nữa; IP khách hàng đi qua một tiêu đề riêng do bộ cân bằng tải ghi là `X-Real-IP`, được đưa vào log và truyền cho PHP qua `fastcgi_param REMOTE_ADDR`. Dòng log ở node và ở bộ cân bằng tải vẫn ghi cùng một địa chỉ khách hàng, nên truy được một yêu cầu xuyên hai tầng, nhưng quyết định cho-phép chỉ dựa trên địa chỉ của bao TCP thật.

`@NODE_NAME@` được thay bằng tên node thật trong `entrypoint.sh` lúc khởi động. Header này là dụng cụ đo của đề tài: không có nó thì không đếm được request rơi vào node nào. Trong hệ thống thật nên tắt.

Hai chi tiết về thứ tự ưu tiên của `location` chỉ lộ ra khi thử bằng request thật.

Đường `/healthz` lúc đầu viết là `try_files $uri /healthz.php =404` đặt trong `location = /healthz`. Cách viết đó cho phép `nginx` trả chính tệp `healthz.php` như một tệp tĩnh, tức là in ra mã nguồn PHP thay vì thực thi nó. Bản hiện tại bỏ `try_files` và khai báo thẳng `fastcgi_param SCRIPT_FILENAME /var/www/public/healthz.php`, nên đường duy nhất tới tệp PHP là qua khối xử lý FastCGI.

Rule chặn tệp cấu hình từng được viết `location ~ ^/(config\.php|\.env)` và đặt sau `location ~ \.php$`. `nginx` xét các regex theo thứ tự xuất hiện trong tệp, nên request tới `/config.php` đã bị khối xử lý `.php` bắt trước và rule chặn không bao giờ chạy, không kèm dấu hiệu nào. Viết lại thành `location = /config.php` thì đúng, vì match tuyệt đối có ưu tiên cao hơn mọi regex. Đây cũng là dạng lỗi im lặng: `nginx` vẫn nhận cấu hình, chỉ là điều khai báo để chặn thì không bao giờ được chọn.

2.4.3 Ứng dụng demo

Ứng dụng viết bằng PHP, gồm trang chủ liệt kê bài viết đọc từ MariaDB, trang đăng nhập, trang thành viên yêu cầu có phiên, trang nhật ký dành cho quản trị viên, và đường `/healthz`.

Vài điểm liên quan tới bảo mật phiên được làm đúng theo giáo trình. Mật khẩu lưu bằng `password_hash` với `bcrypt`, không lưu dạng thuần. Câu truy vấn dùng `prepare` và `bind_param`. Sau khi xác thực thành công, `session_regenerate_id(true)` được gọi để đổi định danh phiên, chặn tấn công cố định phiên. Token chống CSRF sinh bằng `random_bytes(32)` và so bằng `hash_equals`.

Nhật ký đăng nhập ghi vào bảng `login_audit` kèm cột `node_name`. Vì bảng nằm trong cơ sở dữ liệu dùng chung, chuỗi sự kiện không mất đi khi một node bị tắt, khác với ghi ra tệp cục bộ của node.

2.5 Cấu hình bộ cân bằng tải

2.5.1 Khối *upstream*

```
upstream web_cluster {
    ${LB_ALGO_DIRECTIVE}

    server web01:${BACKEND_PORT} max_fails=${MAX_FAILS} fail_timeout=${FAIL_TIMEOUT};
```

```

server web02:${BACKEND_PORT} max_fails=${MAX_FAILS} fail_timeout=${FAIL_TIMEOUT};
server web03:${BACKEND_PORT} max_fails=${MAX_FAILS} fail_timeout=${FAIL_TIMEOUT};

keepalive 32;
}

```

Image nginx chính thức có sẵn cơ chế thay thế biến môi trường vào các file template khi container khởi động, nên thuật toán và các ngưỡng trở thành tham số chạy được mà không phải sửa file cấu hình. Đổi thuật toán chỉ cần khởi động lại lb01 với một biến môi trường khác.

Vòng keepalive 32 giữ lại các kết nối tới node đã mở xong, tránh phải bắt tay TCP lại cho từng yêu cầu.

Bảng 6. Các tham số đưa ra biến môi trường

Biến	Mặc định	Ý nghĩa
LB_ALGO_DIRECTIVE	least_conn;	Dòng khai báo thuật toán trong upstream; để trống nghĩa là round robin
MAX_FAILS	2	Số lần lỗi liên tiếp để đánh dấu node không đạt
FAIL_TIMEOUT	10s	Thời gian node bị loại, đồng thời là cửa sổ đếm lỗi
BACKEND_PORT	80	Cổng node lắng nghe
SESSION_STORE	redis	redis hoặc file, dùng để đối chứng ở Chương 3

2.5.2 Chuyển tiếp và phát hiện lỗi

```

proxy_connect_timeout 2s;
proxy_read_timeout 10s;
proxy_next_upstream error timeout http_500 http_502 http_503 http_504;
proxy_next_upstream_tries 2;

```

proxy_connect_timeout 2s là khoảng thời gian tối đa nginx chờ một node chấp nhận kết nối TCP. proxy_next_upstream liệt kê những trường hợp được phép chuyển sang node khác. Hai tham số này cùng quyết định độ dài của khoảng chờ mà người dùng phải chịu khi node đã chết, đo ở Chương 3.

Vì nginx bản mã nguồn mở không có kiểm tra sức khỏe chủ động, node chỉ bị phát hiện khi có một yêu cầu thật gửi tới và thất bại. Chi phí của cơ chế thụ động này chính là các request phải chờ hết 2 giây.

2.5.3 Định dạng log

```

log_format lb_trace '$remote_addr - [$time_local] "$request" '
                    'status=$status upstream=$upstream_addr '
                    'upstream_status=$upstream_status '
                    'upstream_time=$upstream_response_time '
                    'req_time=$request_time xff="$http_x_forwarded_for"';

```

Biến \$upstream_addr ghi lại mọi node mà nginx đã thử cho một yêu cầu. Một dòng log có hai địa chỉ cách nhau bởi dấu phẩy là bằng chứng của việc chuyển node, và

\$upstream_status cho biết node đầu trả mã gì. Toàn bộ số liệu failover trong Chương 3 lấy từ trường này.

2.6 Kho phiên dùng chung

Phiên được chuyển sang Redis bằng hai dòng khai báo sinh trong `entrypoint.sh`:

```
session.save_handler = redis
session.save_path     = "tcp://redis01:6379?auth=...&database=0&prefix=de07:"
```

Redis chạy với `--requirepass`, `--maxmemory 128mb`, `--maxmemory-policy allkeys-lru` và `--save ""`, tức là dữ liệu phiên chỉ nằm trong bộ nhớ và không ghi đĩa. Đây là lựa chọn đúng cho đối tượng phiên: phiên có vòng đời ngắn, mất thì đăng nhập lại, đổi lại không phải chịu độ trễ ghi đĩa.

Các cờ cookie đặt tại node nên mọi node gửi về cùng một chính sách: `session.cookie_httponly`, `session.cookie_secure`, `session.cookie_samesite = Lax`, `session.use_strict_mode = 1` và `session.sid_length = 32`.

Redis nằm trong mạng backend và không có cổng công bố. Ai đọc được kho này thì giả được mọi phiên đang hoạt động, nên nó phải đứng sau cùng một hàng rào mạng với cơ sở dữ liệu.

2.7 Lớp bảo mật của cụm

2.7.1 Kết thúc TLS và header phản hồi

TLS kết thúc tại `lb01` với `ssl_protocols TLSv1.2 TLSv1.3` và chứng chỉ tự ký khai báo `subjectAltName` gồm cả địa chỉ `192.168.240.10`. Toàn bộ cổng 80 chỉ làm một việc: chuyển hướng 301 sang HTTPS.

```
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header X-Content-Type-Options    "nosniff" always;
add_header X-Frame-Options           "SAMEORIGIN" always;
add_header Content-Security-Policy    "default-src 'self'; style-src 'self' 'unsafe-
inline'; img-src 'self' data;; script-src 'self'" always;
proxy_hide_header Server;
proxy_hide_header X-Powered-By;
```

`server_tokens off` đặt ở khối `http` để bản thân `lb01` cũng không lộ phiên bản. Đoạn từ bộ cân bằng tải về node chạy HTTP thuần trong mạng nội bộ; đây là đánh đổi của việc kết thúc TLS tập trung và được bàn ở mục 2.7.5.

```
lb-lab-03 du header an toan va cookie phien
$ docker compose exec -T client01 curl -sk -D - -o /dev/null https://192.168.240.10/login.php |
sed -E "s/(SID=)[A-Za-z0-9]+/\1d..."
HTTP/2 200
server: nginx
date: Tue, 29 Sep 2026 05:56:36 GMT
content-type: text/html; charset=UTF-8
set-cookie: DE07SID=<da-che>; path=/; secure; HttpOnly; SameSite=Lax
expires: Thu, 19 Nov 1981 08:52:00 GMT
cache-control: no-store, no-cache, must-revalidate
pragma: no-cache
x-node: web01
strict-transport-security: max-age=31536000; includeSubDomains
x-content-type-options: nosniff
x-frame-options: SAMEORIGIN
referrer-policy: strict-origin-when-cross-origin
content-security-policy: default-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' da
ta;; script-src 'self'
exit code: 0
```

Hình 11 Một đáp ứng HTTPS đi qua cụm: header Server không kèm phiên bản, đủ bốn header an toàn, cookie phiên mang cả ba cờ

2.7.2 Giới hạn tần suất

```
limit_req_zone $binary_remote_addr zone=auth:10m rate=5r/s;
limit_req_zone $binary_remote_addr zone=probe:10m rate=5r/s;
limit_conn_zone $binary_remote_addr zone=perip:10m;

location ~ ^/(login|logout)\.php$ {
    limit_req zone=auth burst=5;
    limit_conn perip 20;
    ...
}
```

Giới hạn đặt riêng cho hai trang đăng nhập và đăng xuất, là nơi chịu tấn công dò mật khẩu, để không ảnh hưởng tới các phép đo phân phối tải.

Quá trình triển khai phát hiện một lỗi cấu hình. Location dùng để kiểm chứng giới hạn tần suất ban đầu chỉ có `return 200`, và kết quả là 40 request liên tiếp tất cả đều trả 200, nghĩa là giới hạn không hề chạy. Nguyên nhân: `return` thực thi ở giai đoạn `rewrite`, còn `limit_req` được xét ở giai đoạn `preaccess` muộn hơn, nên nginx trả lời luôn trước khi kịp kiểm tra. Phải chuyển location đó sang `proxy_pass` thì giới hạn mới có tác dụng. Đây là dạng lỗi cấu hình im lặng: tính năng trông như đang bật nhưng thực tế tắt.

Một lỗi khác ở ngay dòng khai báo, thuộc loại ngược lại vì nó khiến nginx từ chối khởi động: từ khóa `nodelay` được viết kèm `limit_req_zone`, trong khi `nodelay` là tham số của `limit_req`. nginx từ chối khởi động và chỉ rõ dòng sai. Hai dạng lỗi này cần phân biệt, vì lỗi thứ nhất mới là lỗi nguy hiểm: hệ thống vẫn chạy, vẫn trả lời đúng, chỉ có biện pháp bảo mật là không còn tác dụng.

2.7.3 Cách ly và IP khách hàng

Hai lớp cách ly chồng lên nhau. Lớp thứ nhất là mạng backend `internal`: `true`: node không có đường ra Internet và không có cổng nào ở máy chủ, nên không thể bị gọi thẳng từ ngoài. Lớp thứ hai nằm ở chính node: `allow 172.20.0.10; deny all;` khiến node chỉ trả lời bộ cân bằng tải, kể cả khi người gọi đã nằm trong mạng backend. Không có lớp thứ hai thì chỉ cần một container khác trong cùng mạng là đọc được dữ liệu của node mà không đi qua bất kỳ kiểm soát nào ở tầng bày.

Lớp thứ hai là quy tắc ở tầng ứng dụng, không phải quy tắc tường lửa ở tầng mạng: nó chặn sau khi kết nối TCP đã thiết lập nên vẫn để lộ việc cổng đang mở. Khi backend trải trên nhiều máy vật lý, cần thêm quy tắc tường lửa theo địa chỉ nguồn và mã hoá cả đoạn từ bộ cân bằng tải về node.

```
lb-lab-15 gọi thang node từ mạng ngoài không được
=== C:/Users/Administrator/Documents/Qoder/2026-09-29/8b86a2bd/de07-web-1b-cluster ===
$ docker compose exec -T client01 curl -s -m 4 -o /dev/null -w "http_code=%{http_code}\n" http://172.20.0.12/healthz 2>&1 | grep ...
http_code=000
curl không nối được đến node, dùng như thiết kế
exit code: 0
```

Hình 12 Gọi thẳng vào node từ mạng phía ngoài: curl không thiết lập được kết nối

```
lb-lab-16 node từ container không phải load b
$ docker compose exec -T web03 curl -s -m 4 -o /dev/null -w "http_code=%{http_code}\n" http://172.20.0.12/healthz
http_code=403
exit code: 0
```

Hình 13 Container khác trong cùng mạng backend gọi tới node bị trả về mã 403

```
lb-lab-17 không gọi trực tiếp được thư viện riêng
=== C:/Users/Administrator/Documents/Qoder/2026-09-29/8b86a2bd/de07-web-1b-cluster ===
$ docker compose exec -T client01 curl -sk -i https://192.168.240.10/lib/bootstrap.php | head -3
HTTP/2 403
server: nginx
date: Tue, 29 Sep 2026 06:02:36 GMT
exit code: 0
```

Hình 14 Đường dẫn vào thư viện riêng của ứng dụng bị node chặn bằng mã 403

2.7.4 Đường kiểm tra sức khỏe và các đường dùng riêng cho phép đo

/healthz trả về đúng hai trường:

```
{ "status": "ok", "node": "web01" }
```

Đường này không mở session, không đọc cơ sở dữ liệu, không in phiên bản PHP hay đường dẫn tuyệt đối. Ở bộ cân bằng tải, /healthz đặt ở cả hai server block để giám sát gọi được qua HTTP lẫn HTTPS mà không bị chuyển hướng.

Riêng cho phép đo hiệu năng ở mục 3.1.3, cấu hình có thêm hai đường nữa.

/lb-only nằm ở bộ cân bằng tải và trả lời ngay tại chỗ, không mở kết nối tới node:

```
location = /lb-only {
    return 200 "lb\n";
}
```

/dbping.php nằm ở ứng dụng trên node, giống /healthz nhưng cộng thêm một truy vấn SELECT COUNT(*) thật tới MariaDB. Độ dài đáp ứng của cả hai đường này như nhau trên cả ba node vì tên node có cùng số ký tự; đây là điều kiện cần để ApacheBench không nhầm một đáp ứng khác độ dài thành một request thất bại.

Hai đường này tồn tại chỉ để đo, không thay thế /healthz làm đường giám sát.

2.7.5 Đánh đổi của kết thúc TLS tập trung

Gom TLS về một chỗ làm chứng chỉ chỉ phải quản lý một nơi, giảm chi phí giải mã ở node và cho phép bộ cân bằng tải đọc nội dung HTTP để định tuyến. Đổi lại, đoạn trong

mạng backend là HTTP thuần, cookie vẫn mang định danh phiên thật nhưng đi không mã hóa trên đoạn đó. Với đề tài này, đoạn trong mạng backend không thể bị nghe từ ngoài vì mạng là internal và không định tuyến, nên đánh đổi chấp nhận được. Khi backend đặt ở máy vật lý khác, phải mã hóa cả hai đầu.

2.8 Hướng loại bỏ điểm hỏng đơn lẻ còn lại

Sau khi triển khai, cụm chịu được một node chết nhưng không chịu được 1b01 chết. Hướng xử lý là hai bộ cân bằng tải tranh nhau một địa chỉ ảo theo VRRP, dùng keepalived, kèm vrrp_script để node nào mất nginx thì tự hạ ưu tiên và nhả địa chỉ ảo.

Cấu hình mẫu để ở thư mục keepalived/ nhưng không bật trong đề tài. Lý do: VRRP chạy trên giao thức 112 với địa chỉ multicast 224.0.0.18, còn Docker Desktop trên Windows không bảo đảm chuyển được multicast giữa các container. Đây là hướng phát triển ghi ở Kết luận.

2.9 Kết chương

Chương này đã triển khai cụm ba máy chủ web với một bộ cân bằng tải nginx, kho phiên Redis dùng chung và một MariaDB dùng chung, tổ chức thành hai mạng trong đó node web không có cổng công bố và không có đường ra Internet. Thuật toán cân bằng tải, ngưỡng phát hiện lỗi và chế độ lưu phiên đều là tham số chạy, cho phép đổi mà không sửa cấu hình. Sáu yêu cầu đặt ra ở mục 2.1 được đưa sang Chương 3 để đo và kiểm chứng bằng số liệu.

CHƯƠNG 3. THỬ NGHIỆM VÀ ĐÁNH GIÁ

3.1 Môi trường và cách tiến hành

3.1.1 Môi trường phần cứng và phần mềm

Bảng 7. Môi trường thử nghiệm

Mục	Giá trị
Máy	MSI Thin GF63 12UC, Intel Core i7-12650H, 16 logical processor
Bộ nhớ	16.085 MB
Hệ điều hành	Windows 11 Pro, bản 10.0.26200
Lớp ảo hóa	WSL 2.7.14, kernel 6.18.33.2, giới hạn 4 GB và 6 CPU qua .wslconfig
Engine container	Docker 29.8.1, Docker Compose v5.5.1
Bộ cân bằng tải	nginx 1.27.5
Node web	Alpine 3.20, nginx và PHP 8.3.15
Kho phiên	Redis 7
Cơ sở dữ liệu	MariaDB 11.4
Máy đo	container client01, curl và ApacheBench 2.3

3.1.2 Cách tiến hành

Mọi phép đo đều chạy từ container `client01`, không phát từ Windows, vì lý do đã nêu ở mục 2.2.3. Mỗi phép đo lưu hai thứ: bảng tổng hợp và log thô, nằm trong thư mục `results/`. Các con số trong chương này đọc lại từ chính log thô, không nhập tay.

Tập lệnh tổng hợp `summarize.awk` đã được đối chiếu trước bằng một file dữ liệu giả có đáp án biết trước: mười mẫu, hai mẫu lỗi nằm ở vị trí 6 và 7, khoảng gián đoạn khai báo 400 ms. Chương trình trả về đúng từng con số, kể cả bảng phân phối node trong đó hai mẫu không đọc được tên node bị đẩy vào ô riêng thay vì bị tính nhầm vào một node thật. Bộ phân tích header cũng thử riêng trên một khối header có sẵn ký tự xuống dòng kiểu CR LF. Nhờ đó các con số trong chương là kết quả đo của hệ thống, không phải sản phẩm của công đoạn tổng hợp.

```

lb-lab-19 ket qua tho lưu trong results
=== C:/Users/Administrator/Documents/Qoder/2026-09-29/8b86a2bd/de07-web-1b-cluster ===
$ find results -type f -not -path 'results/_*' | sed 's|[/\[]*$||' | sort | uniq -c; echo; ls -l
results/bench results/security |...

  1 results
  8 results/bench
 38 results/distribute
 11 results/failover
 52 results/failover/repeat-20260929-144558
 11 results/security
  8 results/session

-rw-r--r-- 1 Administrator 197121 25897 Sep 29 11:04 matrix-run.log
-rw-r--r-- 1 Administrator 197121 4623 Sep 29 11:58 rendered-degrade-20260929-115844.conf

results/security:
total 44
-rw-r--r-- 1 Administrator 197121 1946 Sep 29 09:27 seccheck-20260929-092748.txt
-rw-r--r-- 1 Administrator 197121 2057 Sep 29 11:30 seccheck-20260929-112949.txt
-rw-r--r-- 1 Administrator 197121 2091 Sep 29 11:30 seccheck-20260929-113044.txt
-rw-r--r-- 1 Administrator 197121 2206 Sep 29 12:12 seccheck-20260929-121226.txt
-rw-r--r-- 1 Administrator 197121 2207 Sep 29 12:28 seccheck-20260929-122843.txt
-rw-r--r-- 1 Administrator 197121 2207 Sep 29 12:36 seccheck-20260929-123640.txt
-rw-r--r-- 1 Administrator 197121 2202 Sep 29 12:39 seccheck-20260929-123901.txt
-rw-r--r-- 1 Administrator 197121 1932 Sep 29 12:52 seccheck-20260929-125223.txt
-rw-r--r-- 1 Administrator 197121 1932 Sep 29 13:01 seccheck-20260929-130135.txt
-rw-r--r-- 1 Administrator 197121 1955 Sep 29 14:43 seccheck-20260929-144318.txt
-rw-r--r-- 1 Administrator 197121 1955 Sep 29 15:25 seccheck-20260929-152524.txt

exit code: 0

```

Hình 15 Danh sách file kết quả của lượt chạy được giữ lại làm nguồn cho mọi số liệu trong chương

3.1.3 Lệnh đo và cách đọc kết quả

Toàn bộ phép đo hiệu năng dùng ApacheBench. Một lệnh đầy đủ có dạng:

```
| ab -k -n 3000 -c 20 -q https://192.168.240.10/healthz
```

Bảng 8. Ý nghĩa từng tham số của lệnh đo

Tham số	Ý nghĩa	Lý do chọn giá trị này
-n 3000	Tổng số yêu cầu gửi đi	Đủ dài để một vài yêu cầu chậm không làm lệch trung bình, nhưng vẫn kết thúc trong vài giây để chạy được nhiều lượt
-c 20	Số kết nối giữ đồng thời lúc nào cũng mở	Tạo ra trạng thái có nhiều kết nối đang mở, tức điều kiện cần để least connection khác round robin
-k	Bật keep-alive, nhiều yêu cầu dùng lại một kết nối TCP	Khi bật thì đo được tốc độ phục vụ yêu cầu; khi tắt thì phần lớn thời gian là bắt tay TLS, đo được tốc độ dựng kết nối
-q	Không in tiến trình	Để kết quả ghi thẳng vào file
đường dẫn	Endpoint bị đo	Chọn ba endpoint khác nhau để tách chi phí từng tầng, giải thích ở dưới

Tham số -c 20 là số kết nối luôn mở đồng thời, không phải số người dùng. Con số "Time per request" mà ab in ra là độ trễ trung bình của một yêu cầu trong điều kiện có 20 yêu cầu khác đang bay, không phải độ trễ của một yêu cầu đơn lẻ.

Ba yếu tố phương pháp được áp dụng cho toàn bộ phép đo, đưa vào sau khi các phép thử đầu tiên cho kết quả không ổn định.

Thứ nhất, mỗi ô của ma trận đo chạy 20 lượt độc lập, giữa các lượt nghỉ một giây. Một lượt duy nhất không đủ: hai lượt của cùng một cấu hình `least_conn` trong các phép thử thăm dò đã lệch nhau tới 34 phần trăm. Mỗi lượt cũng đặt giới hạn 240 giây; chế độ `keep-alive` đôi khi khiến ab không tự thoát trên WSL 2, và một lượt treo như vậy nếu không bị cắt sẽ làm hỏng toàn bộ các lượt sau vì chiếm tài nguyên của máy.

Thứ hai, trước 20 lượt đo là một lượt làm nóng không tính vào kết quả. Lượt này để bộ đệm của `nginx`, kho kết nối giữ lại tới node và cache của `MariaDB` đi vào trạng thái ổn định, tránh việc lượt đầu tiên luôn chậm hơn các lượt sau một cách hệ thống.

Thứ ba, kết quả trình bày dưới dạng trung bình kèm khoảng tin cậy 95% tính theo phân phối Student, và có phép kiểm Welch khi so sánh hai thuật toán. Trung bình một mình không nói được gì; phải so độ rộng khoảng tin cậy với độ lớn của hiệu ứng thì mới kết luận được hai cấu hình khác nhau hay chỉ khác nhau do nhiễu.

Ký hiệu dùng chung trong các bảng của mục 3.2 và 3.5: dấu " $\pm$ " sau giá trị trung bình là nửa khoảng tin cậy 95% (CI) của trung bình đó; CV là hệ số biến thiên, tức độ lệch chuẩn chia cho trung bình, đo độ nhiễu của chính phép đo; P50, P95, P99 và P100 là các phân vị thời gian đáp ứng tính trên từng request trong một lượt đo, trong đó P50 mô tả request ở giữa, P99 mô tả 1% request chậm nhất và P100 là request chậm nhất của lượt đo.

`Keep-alive` được đưa vào làm một chiều của ma trận thay vì chọn một giá trị cố định, vì hai chế độ này đo hai đại lượng khác nhau.

Để tách chi phí của bộ cân bằng tải khỏi chi phí của ứng dụng, mỗi lượt đo trên một endpoint khác nhau:

Bảng 9. Ba endpoint dùng để tách chi phí theo tầng

Endpoint	Đường đi của yêu cầu	Tầng được thêm vào
<code>/lb-only</code>	ab tới <code>lb01</code> , <code>nginx</code> trả lời tại chỗ	chỉ bộ cân bằng tải
<code>/healthz</code>	<code>lb01</code> sang node, node gọi <code>php-fpm</code>	thêm <code>nginx</code> và <code>php-fpm</code> trên node
<code>/dbping.php</code>	nhu trên, cộng một truy vấn <code>SELECT COUNT</code>	thêm <code>MariaDB</code>

Hiệu số thời gian giữa hai dòng liên tiếp chính là chi phí của tầng vừa thêm vào, với điều kiện các yếu tố khác không đổi.

Ba thuật toán nhân ba endpoint nhân hai chế độ kết nối thành 18 ô. Mỗi ô 20 lượt đo cùng một lượt làm nóng, tổng cộng 360 lượt đo và 18 lượt làm nóng, ghi vào `results/bench/matrix-<Thời gian>.csv`. Hai phép đo khác dùng cùng giao thức nhưng trả lời câu hỏi khác: `results/distribute/skew-<Thời gian>.csv` chứa 60 lượt đo phân phối cho mục 3.2.2, còn `results/bench/degrade-<Thời gian>.csv` chứa 80 lượt đo (kèm 4 lượt làm nóng) so sánh cụm ba node với cụm hai node ở mục 3.5.4, kèm

degrade-nodes-<Thời gian>.csv ghi số request mà từng node thực sự phục vụ trong mỗi kích thước. Chương trình tổng hợp của cả ba tập là analyze_bench.py; mọi con số hiệu năng trong mục 3.2 và mục 3.5 chép từ đầu ra của đúng chương trình đó, và cũng đầu ra ấy được chụp lại thành hình.

3.2 Khả năng phân phối yêu cầu

3.2.1 Phép đo tuần tự

Gửi 300 request lần lượt tới /healthz, mỗi request một tiến trình curl riêng, đếm giá trị header X-Node ở đáp ứng.

Bảng 10. Phân phối 300 request gửi tuần tự

Thuật toán	web01	web02	web03	Request thiếu header
round robin	104	98	98	0
least_conn	104	98	98	0
ip_hash	300	0	0	0

Kết quả xác nhận cụm có phân phối được yêu cầu và ip_hash dồn toàn bộ về một node khi chỉ có một địa chỉ nguồn. Nhưng hai dòng đầu gây nghi ngờ: không những cùng tổng số, hai phép chạy còn cho **chuỗi 300 phần tử giống hệt nhau từng vị trí**.

Nghi vấn đầu tiên là thuật toán chưa thực sự đổi. Để loại trừ, mỗi lần chạy đã lưu lại đúng khối upstream mà nginx đang dùng. Ba file cấu hình khác nhau thật: vòng round robin có dòng comment thay cho directive, vòng least_conn có least_conn;, vòng ip_hash có ip_hash;.

Vậy đây là hành vi đúng của hệ thống, và nó chỉ ra một hạn chế của cách đo. least connection chọn node có ít kết nối đang mở nhất. Khi request gửi tuần tự, tại thời điểm chọn luôn không có kết nối nào đang mở, mọi node đều hòa nhau, và nginx phân xử theo đúng thứ tự vòng. Nói cách khác, với tải tuần tự thì least connection suy biến thành round robin, phép đo này không phân biệt được hai thuật toán.

```

lb-lab-04 phân phối tuần tự round robin
=== C:/Users/Administrator/Documents/Qoder/2026-09-29/8b86a2bd/de07-web-lb-cluster ===
$ LB_ALGO_DIRECTIVE="# round_robin: không khai báo directive" docker compose up -d --force-recre
ate --no-deps lb01 > /dev/null 2>...

Gửi 300 request tới https://192.168.240.10/healthz (thuật toán: round_robin)
algorithm=round_robin
requests=300
missing_node_header=0
distribution:
104 web01
98 web02
98 web03
exit code: 0

```

Hình 16 Phép đo tuần tự với round robin

```

lb-lab-05 phân phối tuần tự least_conn, cùng một c
=== C:/Users/Administrator/Documents/Qoder/2026-09-29/8b86a2bd/de07-web-lb-cluster ===
$ LB_ALGO_DIRECTIVE="least_conn;" docker compose up -d --force-recreate --no-deps lb01 > /dev/nu
11 2>&1; sleep 6; docker compose ...

Gui 300 request tới https://192.168.240.10/healthz (thuật toán: least_conn)
algorithm=least_conn
requests=300
missing_node_header=0
distribution:
104 web01
98 web02
98 web03
exit code: 0

```

Hình 17 Phép đo tuần tự với `least_conn`, cho cùng chuỗi phân phối như round robin

```

lb-lab-06 ip_hash đơn toàn bộ về một node
=== C:/Users/Administrator/Documents/Qoder/2026-09-29/8b86a2bd/de07-web-lb-cluster ===
$ LB_ALGO_DIRECTIVE="ip_hash;" docker compose up -d --force-recreate --no-deps lb01 > /dev/null
2>&1; sleep 6; docker compose exe...

Gui 300 request tới https://192.168.240.10/healthz (thuật toán: ip_hash)
algorithm=ip_hash
requests=300
missing_node_header=0
distribution:
300 web01
exit code: 0

```

Hình 18 Phép đo tuần tự với `ip_hash`, toàn bộ request đi về một node

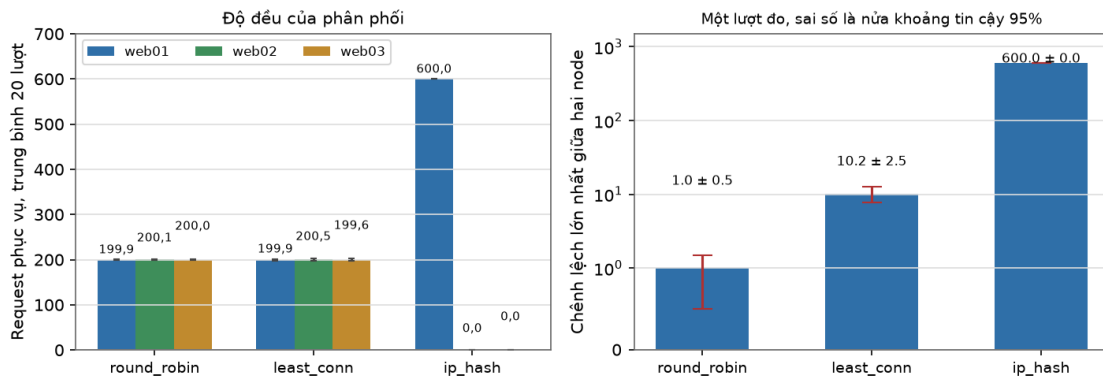
3.2.2 Phép đo đồng thời hai mươi lượt

Để tạo ra trạng thái có nhiều kết nối mở cùng lúc, dùng ApacheBench gửi 600 request qua 20 kết nối song song và đếm node phục vụ từ chính access log của bộ cân bằng tải. Trường `$upstream_addr` trong log ghi lại địa chỉ node đã trả lời, nên phép đếm này không phụ thuộc vào việc node có gửi header đo hay không. Mỗi thuật toán chạy 20 lượt độc lập, sau một lượt làm nóng không tính vào kết quả, theo đúng giao thức đã nêu ở mục 3.1.3.

Bảng 11. Phân phối 600 request gửi đồng thời qua 20 kết nối, tổng hợp từ 20 lượt độc lập cho mỗi thuật toán

Thuật toán	web01	web02	web03	Độ lệch lớn nhất giữa hai node trong một lượt	Request phải thử lại ở node khác
round robin	199,9 ± 0,3	200,1 ± 0,3	200,0 ± 0,3	1,0 ± 0,5	0
least_conn	199,9 ± 1,9	200,5 ± 2,9	199,6 ± 2,3	10,2 ± 2,5	0
ip_hash	600,0 ± 0,0	0,0 ± 0,0	0,0 ± 0,0	600,0 ± 0,0	0

Mỗi ô là trung bình của 20 lượt kèm theo nửa khoảng tin cậy 95%. Cột cuối đếm số dòng log có hai địa chỉ upstream, tức những request bị chuyển sang node khác sau khi thử thất bại; mỗi thuật toán gồm 20 lượt, tức 12000 request; cả ba thuật toán là 36000 request, và không có request nào thuộc loại đó.



Hình 19 Trái: số request trung bình mỗi node theo thuật toán, sai số là nửa khoảng tin cậy 95%. Phải: độ lệch lớn nhất giữa hai node đo trong cùng một lượt

```

lib-lab-07 đo lệch phân phối 20 lượt
=== C:/Users/Administrator/Documents/Qoder/2026-09-29/8b86a2bd/de07-web-lb-cluster ===
$ PYTHONUTF8=1 python REPORT/analyze_bench.py skew

== DO LECH PHAN PHOI, 20 LUONG MOI THUT TOAN ==
thuat toan  n web01 (± CI) web02 (± CI) web03 (± CI) lech max thu lai
round_robin 20 199,9 ± 0,3 200,1 ± 0,3 200,0 ± 0,3 1,0 0
least_conn  20 199,9 ± 1,9 200,5 ± 2,9 199,6 ± 2,3 10,2 0
ip_hash     20 600,0 ± 0,0 0,0 ± 0,0 0,0 ± 0,0 600,0 0
exit code: 0

```

Hình 20 Bảng phân phối hai mươi lượt in ra từ analyze_bench.py, đúng nguồn của các con số trong bảng trên

3.2.3 Nhận xét

Độ đều của phân phối xếp hạng được. round robin chia 600 request thành 199,9 / 200,1 / 200,0, và độ lệch lớn nhất giữa hai node trong một lượt đo là $1,0 \pm 0,5$ request. least_conn cho 199,9 / 200,5 / 199,6 với độ lệch $10,2 \pm 2,5$ request. ip_hash dồn toàn bộ 600 request về một node ở cả hai mươi lượt. Ba khoảng 0,5 đến 1,5; 7,7 đến 12,7 và 600 không chồng lên nhau, nên thứ tự round robin đều nhất, least_conn kém hơn, ip_hash không phân phối là kết luận có cơ sở.

Việc least_conn cho cùng một chuỗi phân phối như round robin ở phép đo tuần tự (mục 3.2.1) nhưng kém đều hơn ở phép đo này không phải hai kết quả mâu thuẫn nhau. least connection chọn node có ít kết nối đang mở nhất. Ba node trong cụm chạy cùng một bản ảnh, cùng cấu hình php-fpm và trả về một đáp ứng 30 byte như nhau, nên tại thời điểm chọn, khác biệt giữa các node thường chỉ là không hoặc một kết nối. Thuật toán vì thế quyết định trên một mẫu nhiễu rất nhỏ và tích lũy sai số đó thành độ lệch mười request, trong khi round robin đi theo một vòng đã định sẵn. Đúng như phân tích ở mục 1.5.3, least connection là thuật toán thích nghi và chỉ có lợi thế khi các node không đồng chất hoặc khi tồn tại request chậm; cấu hình của đề tài không có cả hai điều kiện đó.

Số request mỗi node nhận được và tốc độ phục vụ là hai đại lượng khác nhau. Tốc độ của ba thuật toán được đo riêng ở mục 3.5 bằng một ma trận 20 lượt cho mỗi cấu hình.

Với ip_hash, toàn bộ lưu lượng của một địa chỉ nguồn đi về đúng một node, nên node đó vừa là điểm nghẽn vừa là điểm hỏng đơn lẻ. Mục 3.4 sẽ cho thấy dính theo địa chỉ IP

cũng không giữ được phiên khi node đó chết, còn mục 3.5 đo xem việc chỉ dùng một node làm giảm thông qua bao nhiêu khi ứng dụng thật sự phải tính toán.

Yêu cầu thứ nhất của đề bài được thỏa: cụm phân phối được yêu cầu đều trên ba node, và chọn được thuật toán bằng tham số chạy.

3.3 Khả năng đáp ứng khi một máy chủ gặp sự cố

3.3.1 Diễn biến

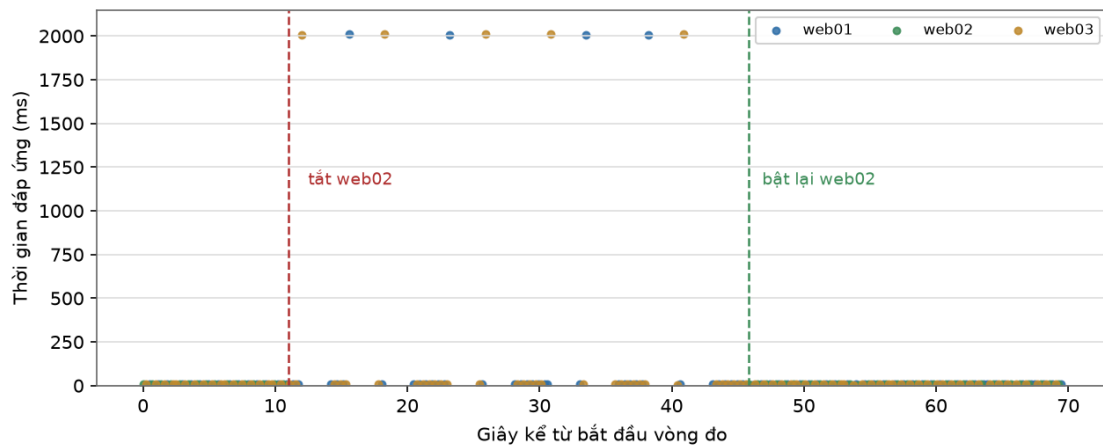
Vòng đo gửi một request, chờ kết quả, nghỉ 0,2 giây rồi gửi tiếp, lặp trong 70 giây, ghi lại thời điểm, mã HTTP, node phục vụ và thời gian đáp ứng của từng mẫu. Chu kỳ thực tế bằng 0,2 giây cộng thời gian đáp ứng, nên nhịp đạt được xấp xỉ 3,2 request mỗi giây và một vòng thu được khoảng 219 mẫu. Ở giây thứ 10, lệnh `docker compose stop web02` tắt hẳn node này. Ở giây thứ 40, `docker compose start web02` bật lại. Toàn bộ vòng đo được lặp lại 12 lần độc lập; mỗi lần lặp giữ lại cả tệp mẫu, bảng tổng hợp và log của lần đó trong `results/failover/repeat-<Thời gian>`.

Bảng 12. Kết quả vòng đo khi tắt web02, trung bình 12 vòng lặp độc lập kèm nửa khoảng tin cậy 95%

Chỉ tiêu	Giá trị
Số mẫu trong vòng đo	$218,9 \pm 13,1$
Số mẫu có mã HTTP khác 200	$0,0 \pm 0,0$
Thời gian đáp ứng trung bình	$90,2 \pm 9,6$ ms
Thời gian đáp ứng lớn nhất	$2026,4 \pm 23,6$ ms
Số mẫu phải chờ quá 1 giây	$8,8 \pm 0,4$
Phân phối trong kỳ đo	web01: $84,6 \pm 4,6$, web02: $49,6 \pm 4,3$, web03: $84,8 \pm 4,6$

Số mẫu phải chờ quá một giây là đại lượng ổn định nhất của phép đo: 12 vòng cho lần lượt 9, 9, 9, 9, 10, 8, 9, 8, 8, 8, 9, 9 mẫu, trung bình 8,8 với nửa khoảng tin cậy 0,4, tức chạy lại thì kết quả dao động trong khoảng 8 đến 10. Chênh lệch giữa hai vòng lặp xa nhất chỉ là hai mẫu. Số mẫu lỗi HTTP bằng 0 ở cả 12 vòng, không có vòng nào ngoại lệ.

Hai đại lượng khác dao động mạnh hơn, và mỗi giá trị trung bình chỉ có nghĩa khi đi kèm khoảng của nó. Số mẫu của một vòng rơi trong dải 174 đến 232 vì vòng đo dừng theo thời gian chứ không theo số request, nên vòng nào gặp nhiều request chậm thì thu ít mẫu hơn. Thời gian đáp ứng trung bình dao động 77 đến 130 ms, hệ số biến thiên 16,7%, chủ yếu do tỉ lệ request trúng lượt thử node chết thay đổi giữa các vòng. Ngược lại, request chậm nhất của mỗi vòng rất ổn định: 2011 đến 2139 ms, tức đúng hai giây chờ của `proxy_connect_timeout` cộng phần dao động nhỏ.



Hình 21 Mỗi chấm là một request trong vòng đo, tô theo node phục vụ. Đường đỏ là thời điểm tắt web02; đường xanh là request đầu tiên web02 phục vụ trở lại sau sự cố, trễ hơn lệnh bật node vài giây

Đồ thị của một vòng đo điển hình cho thấy hai điểm. Không có mẫu nào rơi khỏi mức 200, kể cả ngay sau thời điểm tắt. Chín mẫu bị đẩy lên hơn 2000 ms, nằm rải suốt khoảng thời gian web02 chết. Sau khi web02 bật lại, tất cả trở về nền dưới 20 ms.

```

lb-lab-08 tắt web02 giữa vòng đo
de07-redis01: Up 4 hours (healthy)
de07-web01: Up 44 minutes
de07-web02: Up 14 minutes
de07-web03: Up 44 minutes
[t=10s] docker compose stop web02
[+] stop 1/1
✓ Container de07-web02 Stopped
0.3s
[t=40s] docker compose start web02
[+] start 1/1
✓ Container de07-web02 Started
0.2s

== LOG của load balancer trong khoảng thời gian diễn ra sự cố ==
Đã lưu 25 dòng log vào results/failover/retries-web02-20260929-152350.log (10 dòng access log có
hai địa chỉ upstream, 15 dòng error/warn của nginx)
192.168.240.20 - [29/Sep/2026:08:24:16 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.11:80 upstream_status=504, 200 upstream_time=2.001, 0.001 req_time=2.002 xff=
"_"
192.168.240.20 - [29/Sep/2026:08:24:19 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.13:80 upstream_status=504, 200 upstream_time=2.002, 0.002 req_time=2.003 xff=
"_"
192.168.240.20 - [29/Sep/2026:08:24:24 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.13:80 upstream_status=504, 200 upstream_time=2.003, 0.001 req_time=2.005 xff=
"_"
192.168.240.20 - [29/Sep/2026:08:24:26 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.11:80 upstream_status=504, 200 upstream_time=2.002, 0.001 req_time=2.003 xff=
"_"
192.168.240.20 - [29/Sep/2026:08:24:31 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.11:80 upstream_status=504, 200 upstream_time=2.002, 0.001 req_time=2.003 xff=
"_"
192.168.240.20 - [29/Sep/2026:08:24:34 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.13:80 upstream_status=504, 200 upstream_time=2.002, 0.002 req_time=2.003 xff=
"_"

== KẾT QUẢ DO FAILOVER ==
Tổng số mẫu          : 229
Số mẫu có HTTP != 200 : 0
Thời gian đáp ứng trung bình : 0.0856 s
Thời gian đáp ứng lớn nhất : 2.0097 s

Người dùng không nhận ma lỗi nào trong suốt kỳ đo.
Mau phải chịu thời gian cho vượt 1 s (nginx thu lại node đã chết) : 9

-- Phân phối node trong kỳ đo --
web01      88
web02      50
web03      91
exit code: 0

```

Hình 22 Vòng đo failover: tắt web02 ở giây thứ 10, bật lại ở giây thứ 40, kèm bảng tổng hợp

3.3.2 Bảng chứng trong log của bộ cân bằng tải

Một vòng đo điển hình lưu 25 dòng log ghi trong khoảng thời gian diễn ra sự cố, gồm 15 dòng do nginx ghi vào tệp lỗi và 10 dòng access log có hai địa chỉ upstream. Mười dòng nhiều hơn một dòng so với số mẫu chậm: chín dòng có `upstream_time=2.00x` tức là chờ trọn hai giây, dòng còn lại có `upstream_time=0.001`, `0.001` vì node chết từ chối kết nối ngay bằng RST, request được chuyển node trong một phần nghìn giây nên không vượt ngưỡng một giây. Chín dòng chờ hai giây đúng bằng chín mẫu chậm, và mỗi dòng có dạng:

```
192.168.240.20 - [29/Sep/2026:08:24:02 +0000] "GET /healthz HTTP/2.0"
status=200 upstream=172.20.0.12:80, 172.20.0.13:80
upstream_status=504, 200 upstream_time=2.002, 0.001 req_time=2.004
```

Ba trường này kể lại toàn bộ sự việc. nginx thử node 172.20.0.12, nhận 504 sau 2,002 giây, tức là hết `proxy_connect_timeout`. Sau đó nó chuyển sang 172.20.0.13, node này trả lời trong 0,001 giây. Người dùng cuối nhận một đáp ứng 200 và chỉ phải chờ thêm hai giây.

```
lb-lab-09 log nginx cho thay chuyen node sau 504
-rw-r--r-- 1 Administrator 197121 469 Sep 29 15:11 repeat-20260929-144558.csv
-rw-r--r-- 1 Administrator 197121 768 Sep 29 15:04 repeat-run.log
-rw-r--r-- 1 Administrator 197121 4756 Sep 29 15:04 retries-web02-20260929-150308.log
-rw-r--r-- 1 Administrator 197121 5218 Sep 29 15:10 retries-web02-20260929-150848.log
-rw-r--r-- 1 Administrator 197121 5942 Sep 29 15:25 retries-web02-20260929-152350.log
-rw-r--r-- 1 Administrator 197121 378 Sep 29 15:04 summary-load-web02-20260929-150308.txt
-rw-r--r-- 1 Administrator 197121 378 Sep 29 15:10 summary-load-web02-20260929-150848.txt
-rw-r--r-- 1 Administrator 197121 378 Sep 29 15:25 summary-load-web02-20260929-152350.txt

192.168.240.20 - [29/Sep/2026:08:24:02 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.11:80 upstream_status=502, 200 upstream_time=0.001, 0.001 req_time=0.002 xff=
"_"
192.168.240.20 - [29/Sep/2026:08:24:05 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.13:80 upstream_status=504, 200 upstream_time=2.002, 0.001 req_time=2.003 xff=
"_"
192.168.240.20 - [29/Sep/2026:08:24:09 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.11:80 upstream_status=504, 200 upstream_time=2.002, 0.001 req_time=2.003 xff=
"_"
192.168.240.20 - [29/Sep/2026:08:24:11 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.13:80 upstream_status=504, 200 upstream_time=2.002, 0.002 req_time=2.001 xff=
"_"
192.168.240.20 - [29/Sep/2026:08:24:16 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.11:80 upstream_status=504, 200 upstream_time=2.001, 0.001 req_time=2.002 xff=
"_"
192.168.240.20 - [29/Sep/2026:08:24:19 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.13:80 upstream_status=504, 200 upstream_time=2.002, 0.002 req_time=2.003 xff=
"_"
192.168.240.20 - [29/Sep/2026:08:24:24 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.13:80 upstream_status=504, 200 upstream_time=2.003, 0.001 req_time=2.005 xff=
"_"
192.168.240.20 - [29/Sep/2026:08:24:26 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.11:80 upstream_status=504, 200 upstream_time=2.002, 0.001 req_time=2.003 xff=
"_"
192.168.240.20 - [29/Sep/2026:08:24:31 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.11:80 upstream_status=504, 200 upstream_time=2.002, 0.001 req_time=2.003 xff=
"_"
192.168.240.20 - [29/Sep/2026:08:24:34 +0000] "GET /healthz HTTP/2.0" status=200 upstream=172.20
.0.12:80, 172.20.0.13:80 upstream_status=504, 200 upstream_time=2.002, 0.002 req_time=2.003 xff=
"_"
exit code: 0
```

Hình 23 Các dòng access log tương ứng các mẫu chậm, mỗi dòng có hai địa chỉ upstream và cặp mã 504 rồi 200

3.3.3 Vì sao không có request nào lỗi

Ba cấu hình phối hợp tạo ra kết quả đó. `proxy_connect_timeout 2s` chặn thời gian chờ một node chết, thay vì để mặc định 60 giây. `proxy_next_upstream error timeout http_502 http_503 http_504` cho phép chuyển node khi thử thất bại. `proxy_next_upstream_tries 2` giới hạn số lần thử để một request không đi vòng hết cả cụm.

Chi phí phải trả là hai giây chờ ở những request trúng lượt thử lại. Chín request chậm nằm trải trong 29 giây, cách nhau trung bình 3,6 giây, ngắn nhất 2 giây và dài nhất 5 giây. Khoảng cách này ngắn hơn nhiều chu kỳ `fail_timeout=10s` đã khai trong cấu hình. Lý do là mỗi request trúng lượt thử đều được tính là một lần thất bại, và vì máy đo gửi liên tục nên

ngưỡng `max_fails=2` được tích lại nhanh hơn một chu kỳ hết hạn. Quy luật tương tác giữa ba tham số `max_fails`, `fail_timeout` và `proxy_next_upstream` cần một phép đo riêng; con số ghi ở đây là quan sát trực tiếp trên log của vòng đo.

Đây chính là đặc trưng của kiểm tra sức khỏe thụ động đã nêu ở mục 1.6. nginx bản mã nguồn mở không gửi yêu cầu thăm dò nền, nên nó chỉ biết node hỏng sau khi một người dùng thật phải chịu hậu quả. Nếu dùng HAProxy với option `httpchk`, node chết bị phát hiện mà không cần request thật, và khoảng chín mẫu chậm đó sẽ không tồn tại. Đổi lại phải thêm một thành phần và một lớp cấu hình.

Yêu cầu thứ hai của đề bài được thỏa: khi một máy chủ gặp sự cố, cụm vẫn đáp ứng, không cần can thiệp thủ công.

3.4 Tính liên tục của phiên làm việc

Phép đo này trả lời câu hỏi: nếu node đã phục vụ đăng nhập chết thì người dùng có mất phiên không. Cách làm: đăng nhập, ghi lại node đã phục vụ, tắt đúng node đó, rồi gọi `/member.php` mười lần bằng cookie cũ.

Hai phương án chạy với hai cấu hình khác nhau, và sự khác nhau đó là nội dung của phép đo chứ không phải thiếu sót. Phương án redis dùng `least_conn` vì phiên nằm ngoài node nên được phép chọn tự do. Phương án file buộc phải dùng `ip_hash`, vì nếu không thì phiên đã mất ngay từ bước đăng nhập do token chống CSRF nằm ở node khác.

Bảng 13. Giữ phiên sau khi tắt node đã phục vụ đăng nhập

Phương án	Thuật toán	Node đăng nhập	Node bị tắt	Lần gọi giữ được phiên	Lần gọi bị đá về đăng nhập
redis	<code>least_conn</code>	web01	web01	10 / 10	0 / 10
file	<code>ip_hash</code>	web01	web01	0 / 10	10 / 10

Ở phương án redis, mười lần gọi sau đó chia cho hai node còn lại theo tỉ lệ 7/3, tất cả đều trả 200 và vẫn hiển thị đúng tên người dùng. Ở phương án file, `ip_hash` dồn cả mười lần gọi về cùng một node vì chỉ còn node đó sống, và cả mười lần đều nhận 302 chuyển về trang đăng nhập.

Kết quả này khẳng định nhận định ở mục 1.7: ngay cả khi dùng `ip hash` để dính người dùng vào một node, việc mất node vẫn làm mất phiên. Sticky session chỉ giải quyết được vấn đề khi mọi node còn sống; nó không phải là cơ chế chịu lỗi. Chỉ đưa dữ liệu phiên ra ngoài node mới giải quyết được.

Yêu cầu thứ ba của đề bài được thỏa.

```
lb-lab-10 redis giu duoc phien sau khi tat node
=== C:/Users/Administrator/Documents/Qoder/2026-09-29/8b86a2bd/de07-web-lb-cluster ===
$ bash scripts/session_test.sh redis

=====
Che do luu session: SESSION_STORE=redis
=====

Thuat toan load balancer: least_conn;
# session_test mode=redis stamp=20260929-125936
# load_balancer_algorithm=least_conn;

## Buoc 1. Dang nhap

## Buoc 2. Tat dung node vua phuc vu dang nhap: web01
Container de07-web01 Stopping
Container de07-web01 Stopped

## Buoc 3. Goi lai /member.php 10 lan bang cookie cu
1 code=200 node=web02
2 code=200 node=web02
3 code=200 node=web02
4 code=200 node=web02
5 code=200 node=web02
6 code=200 node=web03
7 code=200 node=web02
8 code=200 node=web03
9 code=200 node=web02
10 code=200 node=web03

Tong ket: 200_giu_duoc_phien=10 302_mat_phien=0 khac=0

## Buoc 4. Bat lai web01
Container de07-web01 Starting
Container de07-web01 Started

## Buoc 5. Tra cum ve cau hinh mac dinh (redis + least_conn)
Da luu ket qua vao results/session/session-redis-20260929-125936.txt
exit code: 0
```

Hình 24 Phương án redis: tắt đúng node vừa phục vụ đăng nhập, mười lần gọi sau vẫn giữ phiên

```
lb-lab-11 file mat phien sau khi tat cung node do
=== C:/Users/Administrator/Documents/Qoder/2026-09-29/8b86a2bd/de07-web-lb-cluster ===
$ bash scripts/session_test.sh file; LB_ALGO_DIRECTIVE="least_conn;" docker compose up -d --force-recreate --no-deps lb01 > /dev/...

=====
Che do luu session: SESSION_STORE=file
=====

Thuat toan load balancer: ip_hash;
# session_test mode=file stamp=20260929-130031
# load_balancer_algorithm=ip_hash;

## Buoc 1. Dang nhap

## Buoc 2. Tat dung node vua phuc vu dang nhap: web01
Container de07-web01 Stopping
Container de07-web01 Stopped

## Buoc 3. Goi lai /member.php 10 lan bang cookie cu
1 code=302 node=web03 redirect=/login.php
2 code=302 node=web03 redirect=/login.php
3 code=302 node=web03 redirect=/login.php
4 code=302 node=web03 redirect=/login.php
5 code=302 node=web03 redirect=/login.php
6 code=302 node=web03 redirect=/login.php
7 code=302 node=web03 redirect=/login.php
8 code=302 node=web03 redirect=/login.php
9 code=302 node=web03 redirect=/login.php
10 code=302 node=web03 redirect=/login.php

Tong ket: 200_giu_duoc_phien=0 302_mat_phien=10 khac=0

## Buoc 4. Bat lai web01
Container de07-web01 Starting
Container de07-web01 Started

## Buoc 5. Tra cum ve cau hinh mac dinh (redis + least_conn)
Da luu ket qua vao results/session/session-file-20260929-130031.txt
exit code: 0
```

Hình 25 Phương án file: cùng kịch bản, cả mười lần gọi bị trả về trang đăng nhập

3.5 Hiệu năng

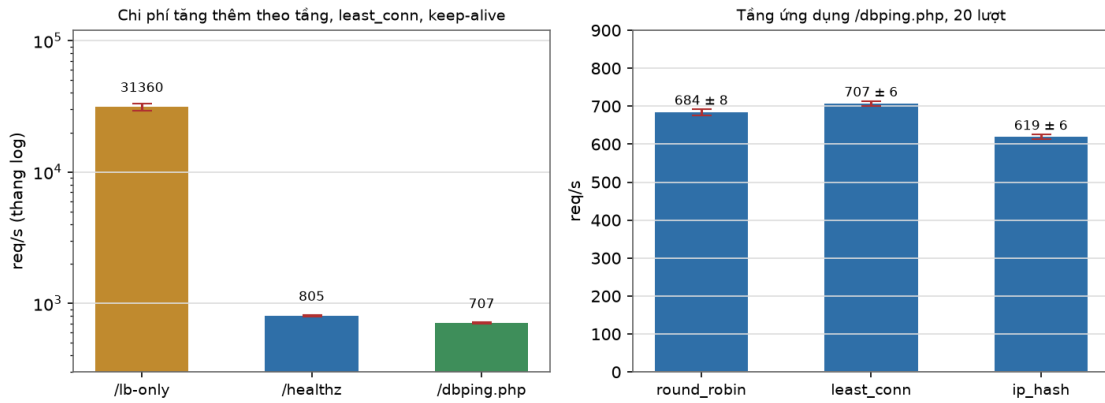
Mục này tách ba chuyện thường bị gộp làm một: bộ cân bằng tải tự nó phục vụ được bao nhiêu yêu cầu, chi phí của từng tầng nằm ở đâu, và ba thuật toán có khác nhau về thông

qua hay không. Nguồn số liệu là `results/bench/matrix-20260929-103024.csv`: 18 ô, mỗi ô 20 lượt đo 3000 request với 20 kết nối song song, đã loại một lượt làm nóng.

3.5.1 Thông qua theo thuật toán và theo tầng phản hồi

Bảng 14. Thông qua trung bình của 20 lượt đo cho mỗi cặp thuật toán và endpoint. Cột CV là hệ số biến thiên, P50 và P99 tính trên từng request

Tầng phản hồi	Kết nối	Thuật toán	req/s $\pm$ 95% CI	CV	P50	P99
/lb-only	không giữ kết nối	round robin	914,3 $\pm$ 23,0	5,4%	22 ms	36 ms
/lb-only	không giữ kết nối	least_conn	918,9 $\pm$ 14,8	3,4%	21 ms	35 ms
/lb-only	không giữ kết nối	ip_hash	909,4 $\pm$ 18,0	4,2%	22 ms	36 ms
/lb-only	keep-alive	round robin	28869,2 $\pm$ 3048,1	22,6%	0 ms	2 ms
/lb-only	keep-alive	least_conn	31360,0 $\pm$ 1922,1	13,1%	0 ms	2 ms
/lb-only	keep-alive	ip_hash	30496,1 $\pm$ 2262,6	15,9%	0 ms	2 ms
/healthz	không giữ kết nối	round robin	759,0 $\pm$ 24,4	6,9%	25 ms	44 ms
/healthz	không giữ kết nối	least_conn	791,4 $\pm$ 24,1	6,5%	24 ms	42 ms
/healthz	không giữ kết nối	ip_hash	854,7 $\pm$ 22,3	5,6%	23 ms	37 ms
/healthz	keep-alive	round robin	848,0 $\pm$ 12,7	3,2%	23 ms	38 ms
/healthz	keep-alive	least_conn	805,0 $\pm$ 8,3	2,2%	24 ms	40 ms
/healthz	keep-alive	ip_hash	817,2 $\pm$ 15,3	4,0%	24 ms	41 ms
/dbping.php	không giữ kết nối	round robin	693,4 $\pm$ 6,3	1,9%	28 ms	46 ms
/dbping.php	không giữ kết nối	least_conn	702,9 $\pm$ 13,1	4,0%	28 ms	44 ms
/dbping.php	không giữ kết nối	ip_hash	621,5 $\pm$ 6,6	2,3%	31 ms	52 ms
/dbping.php	keep-alive	round robin	683,5 $\pm$ 8,4	2,6%	29 ms	46 ms
/dbping.php	keep-alive	least_conn	707,4 $\pm$ 6,4	1,9%	28 ms	45 ms
/dbping.php	keep-alive	ip_hash	619,4 $\pm$ 6,5	2,2%	32 ms	52 ms



Hình 26 Trái: thông qua theo tầng trên thang logarit, sai số là nửa khoảng tin cậy 95%. Phải: cùng phép đo ở tầng ứng dụng, nơi công việc thật của node xuất hiện

```

lb-lab-13 ma tran 18 o thong qua
=== C:/Users/Administrator/Documents/Ooder/2026-09-29/8b86a2bd/de07-web-lb-cluster ===
$ PYTHONUTF8=1 python REPORT/analyze_bench.py matrix | sed -n "1,21p"

nguồn: matrix-20260929-103024.csv (360 luong do, 18 o)
=====
thuat toan  endpoint  ket noi  req/s  ± CI  CV %  P50  P99
round_robin lb-only cold 914,3 23,0 5,4 22 36
least_conn lb-only cold 918,9 14,8 3,4 21 35
ip_hash lb-only cold 909,4 18,0 4,2 22 36
round_robin lb-only keepalive 28869,2 3048,1 22,6 0 2
least_conn lb-only keepalive 31360,0 1922,1 13,1 0 2
ip_hash lb-only keepalive 30496,1 2262,6 15,9 0 2
round_robin healthz cold 759,0 24,4 6,9 25 44
least_conn healthz cold 791,4 24,1 6,5 24 42
ip_hash healthz cold 854,7 22,3 5,6 23 37
round_robin healthz keepalive 848,0 12,7 3,2 23 38
least_conn healthz keepalive 805,0 8,3 2,2 24 40
ip_hash healthz keepalive 817,2 15,3 4,0 24 41
round_robin dbping.php cold 693,4 6,3 1,9 28 46
least_conn dbping.php cold 702,9 13,1 4,0 28 44
ip_hash dbping.php cold 621,5 6,6 2,3 31 52
round_robin dbping.php keepalive 683,5 8,4 2,6 29 46
least_conn dbping.php keepalive 707,4 6,4 1,9 28 45
ip_hash dbping.php keepalive 619,4 6,5 2,2 32 52

exit code: 0

```

Hình 27 Bảng ma trận 18 ô in ra từ chương trình tổng hợp, đúng nguồn của các con số trong mục này

Kết quả của bảng cho thấy ba điểm.

Thứ nhất, ở đường /lb-only, bật keep-alive đòi thông qua từ 914 lên 28869 req/s, gấp 31,6 lần. Cùng một bộ cân bằng tải, cùng một phần cứng, chỉ khác việc có dựng lại kết nối TCP và bắt tay TLS cho mỗi yêu cầu hay không. Khi tắt keep-alive thì phép đo đang đo tốc độ dựng kết nối, không phải tốc độ phục vụ yêu cầu, và cả ba thuật toán cho kết quả không phân biệt được (mục 3.5.3).

Thứ hai, thông qua của bản thân bộ cân bằng tải cao hơn thông qua của cả cụm khi có một node tham gia khoảng 39 lần: 31360 req/s cho riêng lb01 so với 805 req/s khi mỗi yêu cầu phải đi tiếp sang node. Nói cách khác, ở cấu hình này cỗ chai không nằm ở việc chọn node mà nằm ở đoạn từ bộ cân bằng tải tới ứng dụng. Con số 800 req/s là giới hạn của ba node nginx cộng php-fpm trên một máy tính, không phải giới hạn của nginx với vai trò cân bằng tải.

Thứ ba, hệ số biến thiên nội bộ mỗi thuật toán nhỏ hơn nhiều so với các phép đo bốn lượt trước đây: từ 1,9% đến 6,9% ở hai tầng có proxy, riêng tầng /1b-only keep-alive vẫn còn 13% đến 23% vì ở tốc độ 30000 req/s, độ nhiễu của lịch trình tiến trình chiếm tỷ lệ lớn.

3.5.2 Bậc thang chi phí giữa các tầng

Bảng dưới lấy cùng số liệu trên nhưng đổi đơn vị thành thời gian cho một request, tính bằng cách nhân số kết nối song song với 1000 rồi chia cho thông qua, đúng như cách ApacheBench in dòng "Time per request (mean)". Đây là độ trễ trung bình khi đang có 20 yêu cầu bay đồng thời, không phải độ trễ của một người dùng đơn lẻ; độ trễ từng request là các cột P50 và P99 ở bảng trên.

Bảng 15. Chi phí cộng thêm của từng tầng, đo với least_conn

Chế độ kết nối	Tầng	req/s	ms cho một request	Cộng thêm so với tầng trước
không giữ kết nối	riêng bộ cân bằng tải	918,9	21,8	-
không giữ kết nối	thêm nginx và php-fpm trên node	791,4	25,3	3,5 ms
không giữ kết nối	thêm một truy vấn MariaDB	702,9	28,5	3,2 ms
keep-alive	riêng bộ cân bằng tải	31360,0	0,6	-
keep-alive	thêm nginx và php-fpm trên node	805,0	24,8	24,2 ms
keep-alive	thêm một truy vấn MariaDB	707,4	28,3	3,5 ms

Hai dòng keep-alive cho thấy rõ nhất vị trí của chi phí: 24,2 ms trong tổng số 28,3 ms của một yêu cầu ứng dụng thật là chi phí của việc chuyển yêu cầu sang node và chờ php-fpm trả lời, gấp bảy lần chi phí của một truy vấn cơ sở dữ liệu và gấp bốn mươi lần chi phí tự thân của bộ cân bằng tải ở cùng chế độ kết nối. Một truy vấn SELECT COUNT chỉ cộng thêm 3,5 ms. Do đó điểm cần tối ưu nằm ở đoạn từ bộ cân bằng tải tới ứng dụng, không nằm ở thuật toán chọn node.

Ở chế độ không giữ kết nối thì bức tranh ngược lại: 21,8 ms trong 28,5 ms thuộc về việc dựng kết nối và bắt tay TLS, phần proxy chỉ còn 3,5 ms và phần cơ sở dữ liệu 3,2 ms. Cũng một hệ thống, hai câu trả lời khác nhau, tùy ở chỗ người đo có bật keep-alive hay không.

3.5.3 Ba thuật toán có khác nhau về thông qua không

Bảng 16. Kiểm định Welch so với round robin trên cùng endpoint và cùng chế độ kết nối, mỗi bên 20 mẫu

Endpoint	Kết nối	So sánh	Chênh lệch	p	Kết luận ở mức 5%
/lb-only	không giữ kết nối	least_conn	+0,5%	0,7271	không có ý nghĩa thống kê
/lb-only	không giữ kết nối	ip_hash	-0,5%	0,7228	không có ý nghĩa thống kê
/lb-only	keep-alive	least_conn	+8,6%	0,1577	không có ý nghĩa thống kê
/lb-only	keep-alive	ip_hash	+5,6%	0,3758	không có ý nghĩa thống kê
/healthz	không giữ kết nối	least_conn	+4,3%	0,0551	không có ý nghĩa thống kê
/healthz	không giữ kết nối	ip_hash	+12,6%	0,0000	có ý nghĩa thống kê
/healthz	keep-alive	least_conn	-5,1%	0,0000	có ý nghĩa thống kê
/healthz	keep-alive	ip_hash	-3,6%	0,0025	có ý nghĩa thống kê
/dbping.php	không giữ kết nối	least_conn	+1,4%	0,1859	không có ý nghĩa thống kê
/dbping.php	không giữ kết nối	ip_hash	-10,4%	0,0000	có ý nghĩa thống kê
/dbping.php	keep-alive	least_conn	+3,5%	0,0000	có ý nghĩa thống kê
/dbping.php	keep-alive	ip_hash	-9,4%	0,0000	có ý nghĩa thống kê

```

lb-lab-20 welch và bậc thang chi phí
=== C:/Users/Administrator/Documents/Qoder/2026-09-29/8b86a2bd/de07-web-lb-cluster ===
$ PYTHONUTF8=1 python REPORT/analyze_bench.py matrix | sed -n "23,45p"

== SO SANH VOI round_robin cùng endpoint và chế độ kết nối ==
lb-only cold round_robin -> least_conn +0,5% p=0,7271 không có ý nghĩa thống kê
lb-only cold round_robin -> ip_hash -0,5% p=0,7228 không có ý nghĩa thống kê
lb-only keepalive round_robin -> least_conn +8,6% p=0,1577 không có ý nghĩa thống kê
lb-only keepalive round_robin -> ip_hash +5,6% p=0,3758 không có ý nghĩa thống kê
healthz cold round_robin -> least_conn +4,3% p=0,0551 không có ý nghĩa thống kê
healthz cold round_robin -> ip_hash +12,6% p=0,0000 có ý nghĩa thống kê
healthz keepalive round_robin -> least_conn -5,1% p=0,0000 có ý nghĩa thống kê
healthz keepalive round_robin -> ip_hash -3,6% p=0,0025 có ý nghĩa thống kê
dbping.php cold round_robin -> least_conn +1,4% p=0,1859 không có ý nghĩa thống kê
dbping.php cold round_robin -> ip_hash -10,4% p=0,0000 có ý nghĩa thống kê
dbping.php keepalive round_robin -> least_conn +3,5% p=0,0000 có ý nghĩa thống kê
dbping.php keepalive round_robin -> ip_hash -9,4% p=0,0000 có ý nghĩa thống kê

== BẬC THANG CHI PHÍ PHẢN HỒI (least_conn) ==
chế độ cold:
riêng cân bằng tải 919 req/s 21,8 ms
+ nginx và php-fpm trên node 791 req/s 25,3 ms (+3,5 ms)
+ một truy vấn MariaDB 703 req/s 28,5 ms (+3,2 ms)
chế độ keepalive:
riêng cân bằng tải 31360 req/s 0,6 ms
+ nginx và php-fpm trên node 805 req/s 24,8 ms (+24,2 ms)
+ một truy vấn MariaDB 707 req/s 28,3 ms (+3,5 ms)

exit code: 0

```

Hình 28 Đoạn in ra của cùng chương trình chứa mười hai phép kiểm định Welch và bậc thang chi phí ở mục 3.5.2

round robin và least_conn không hơn kém nhau ổn định. Sáu ô so sánh giữa hai thuật toán này cho +0,5%, +8,6%, +4,3%, -5,1%, +1,4% và +3,5%; dấu của hiệu ứng đổi chiều giữa hai chế độ kết nối trên cùng một endpoint, nên không thể nói thuật toán nào nhanh hơn. Hai ô có p nhỏ hơn 0,05 chỉ đạt độ lớn 3,5% và 5,1%, nằm trong vùng mà một thay đổi nhỏ của môi trường đo cũng tạo ra được, nên chưa đủ để xếp hạng hai thuật toán này.

ip_hash thì khác, và chiều đổi ngược của nó là chi tiết đáng chú ý nhất. Ở /healthz không giữ kết nối, ip_hash nhanh hơn round robin 12,6% với p nhỏ hơn 0,0001. Ở /dbping.php nó chậm hơn 10,4%, cũng với p nhỏ hơn 0,0001. Nguyên nhân là /healthz chỉ đủ cho php-fpm trả lời một xâu 30 byte, nên một node gánh toàn bộ lưu lượng vẫn theo kịp và còn lợi hơn vì không phải chia kết nối; khi ứng dụng thật sự phải truy vấn cơ sở dữ liệu thì một node không theo kịp hai node kia. Đây chính là lý do phải đo riêng thông qua của bộ cân bằng tải và thông qua của ứng dụng: chọn thuật toán chỉ vì một con số ở đường kiểm tra sức khỏe sẽ dẫn tới quyết định sai ngay khi ứng dụng bắt đầu làm việc.

3.5.4 Hiệu năng khi mất một node

Hai kích thước được đo bằng cùng một giao thức: 20 lượt, mỗi lượt 3000 request với 20 kết nối song song trên đường /healthz, thuật toán least_conn. Giữa hai lượt đo có một giây nghỉ, và trước đó là một lượt làm nóng. Kích thước hai node lấy bằng cách tắt hẳn web03 và chờ quá một chu kỳ fail_timeout để nginx loại node đó khỏi danh sách.

Bảng 17. So sánh hai kích thước cụm, mỗi con số tổng hợp từ 20 lượt đo

Kích thước	Kết nối	req/s ± 95% CI	CV	P50	P95	P100
ba node	không giữ kết nối	779,3 ± 15,4	4,2%	25 ms	37 ms	54 ms
hai node	không giữ kết nối	719,4 ± 49,6	14,7%	23 ms	35 ms	1048 ms

Kích thước	Kết nối	req/s ± 95% CI	CV	P50	P95	P100
ba node	keep-alive	771,4 ± 16,6	4,6%	26 ms	37 ms	52 ms
hai node	keep-alive	848,3 ± 76,8	19,4%	20 ms	28 ms	2016 ms

Bảng chứng rằng phép đo đúng như tên gọi: trong 126000 request của kích thước ba node, phân phối là 42156 / 41962 / 41882 cho web01, web02, web03; trong 126000 request của kích thước hai node, phân phối là 63058 / 62942 / 0, tức web03 phục vụ chính xác không request nào.

Không có request nào trả về mã lỗi ở cả hai kích thước. Thông qua trung bình của hai node thấp hơn ba node 7,7% ở chế độ không giữ kết nối, kiểm định Welch cho $p = 0,0243$. Ở chế độ keep-alive, trung bình của hai node thậm chí cao hơn 10,0% với $p = 0,0536$, tức chưa loại được khả năng hai cấu hình như nhau.

Ba đại lượng sau phản ánh việc mất node, và cả ba đều nằm ngoài giá trị trung bình.

Thứ nhất, độ biến động. Hệ số biến thiên nhảy từ 4,2% lên 14,7% và từ 4,6% lên 19,4%, tức khoảng tin cậy của chính phép đo rộng gấp ba đến năm lần. Một cụm mất node không chậm đi đều đặn; nó trở nên khó đoán.

Thứ hai, đuôi phân phối. P100, tức request chậm nhất trong lượt đo, đi từ 54 ms lên 1048 ms ở chế độ không giữ kết nối và từ 52 ms lên 2016 ms ở chế độ keep-alive. Con số 2016 ms đúng bằng proxy_connect_timeout 2s đã cấu hình: mỗi chu kỳ fail_timeout, nginx cho một request thử lại node đã chết và request đó nằm chờ trọn hai giây trước khi được chuyển sang node khác.

Thứ ba, số request phải thử lại ở node khác: 126 trên 126000 request của kích thước hai node, chiếm 0,1%, trong khi kích thước ba node là 0. Tỷ lệ nhỏ nhưng tập trung hết vào một nhóm người dùng cụ thể, và đó là nhóm cảm thấy hệ thống có vấn đề.

P50 và P100 của cùng một lượt đo cách nhau hàng trăm lần. Nếu chỉ báo cáo trung bình thì kết luận rút ra là mất một node gần như không ảnh hưởng; nếu chỉ nhìn P95 thì hai dòng 35 ms và 28 ms của kích thước hai node lại thấp hơn 37 ms của kích thước ba node, tức mất node làm hệ thống nhanh lên. Mỗi kết luận trên đều lấy một thống kê duy nhất để đại diện cho cả phép đo. Đối với hệ thống cân bằng tải, tác động của việc mất node thể hiện ở phương sai và ở đuôi phân phối nhiều hơn ở giá trị trung bình.

Lý do thông qua trung bình không giảm tương ứng với việc mất 1/3 số node đã giải thích ở mục 3.5.1: trên đường /healthz, giới hạn nằm ở đoạn máy đo tới bộ cân bằng tải chứ không ở node, nên hai node vẫn thừa khả năng đáp ứng. Tồn thất chỉ hiện ra khi node phải làm việc thật, và ma trận ở mục 3.5.1 đã chỉ ra điều đó ở một phía khác: thuật toán dồn toàn bộ lưu lượng vào một node như ip_hash mất 10,4% thông qua khi endpoint phải truy vấn cơ sở dữ liệu.

```

lb-lab-14 suy giam khi mat mot node
=== C:/Users/Administrator/Documents/Qoder/2026-09-29/8b86a2bd/de07-web-lb-cluster ===
$ PYTHONUTF8=1 python REPORT/analyze_bench.py degrade

== SUY GIAM KHI MAT MOT NODE ==
node ket noi n req/s ± CI CV % P50 P95 P99 P100 loi
2-node cold 20 719,4 49,6 14,7 23 35 42 1048 0
2-node keepalive 20 848,3 76,8 19,4 20 28 32 2016 0
3-node cold 20 779,3 15,4 4,2 25 37 44 54 0
3-node keepalive 20 771,4 16,6 4,6 26 37 44 52 0
mat 1/3 node (cold): -7,7% req/s, p=0,0243
mat 1/3 node (keepalive): +10,0% req/s, p=0,0536

node phục vụ trong hai kích thước (bảng chung của phép đo):
3-node web01=42156 web02=41962 web03=41882 thu lai=0 dong log=126000
2-node web01=63058 web02=62942 web03=0 thu lai=126 dong log=126000

exit code: 0

```

Hình 29 Bảng tổng hợp hai kích thước cụm in ra từ cùng chương trình sinh ra bảng trên, kèm dòng đếm node phục vụ

3.6 Kiểm tra cấu hình và một số thuộc tính an toàn

Mục này trả lời một câu hỏi hẹp: những biện pháp bảo mật đã khai báo ở Chương 2 có thực sự có hiệu lực khi có người truy cập từ ngoài hay không. Hình thức là bảng kiểm cấu hình: mỗi hạng mục là một phát biểu chỉ có thể đúng hoặc sai, kiểm bằng một lệnh đọc mã trạng thái hoặc đọc header trả về, chạy tự động từ container máy đo và ghi kết quả vào results/security/. Phạm vi của mục này là kiểm tra cấu hình, không phải thử nghiệm xâm nhập.

Bảng 18. Kết quả kiểm tra cấu hình và thuộc tính an toàn của cụm, theo đúng thứ tự mà sec_check.sh in ra

Nhóm	Kiểm tra	Kết quả
Lộ thông tin	Header Server không kèm phiên bản	Đạt
Lộ thông tin	Không còn X-Powered-By của PHP	Đạt
Header	Strict-Transport-Security có mặt	Đạt
Header	X-Content-Type-Options nosniff	Đạt
Header	Content-Security-Policy	Đạt
Giao thức	Cổng 80 chuyển cường bức sang HTTPS	Đạt
Giao thức	Thương lượng được TLS 1.2 trở lên, thực tế là TLS 1.3	Đạt
Cookie	Phiên có cờ HttpOnly	Đạt
Cookie	Phiên có cờ Secure	Đạt
Cookie	Phiên có SameSite	Đạt
Kiểm soát truy cập	Không gọi trực tiếp được thư mục lib của ứng dụng	Đạt
Lộ thông tin	/healthz chỉ trả về trạng thái và tên node	Đạt

Nhóm	Kiểm tra	Kết quả
Chống lạm dụng	Giới hạn tần suất chặn request gửi dồn dập	Đạt
Kiểm soát truy cập	Không đọc được file cấu hình của ứng dụng	Đạt
Cách ly	Không gọi được node web từ mạng phía ngoài	Đạt
Cách ly	Node từ chối container khác trong cùng mạng backend	Đạt
Cách ly	Node không có đường ra Internet	Đạt
Log	Node ghi đúng địa chỉ khách hàng sau cân bằng tải	Đạt
Nội dung	Mọi node trả về cùng một nội dung	Đạt

Mười chín hạng mục đều cho kết quả đúng như cấu hình đã khai báo; không có hạng mục thất bại và không còn hạng mục nào phải ghi nhận. Ba kết quả cụ thể như sau.

Hạng mục giới hạn tần suất từng báo đạt một cách sai lệch. Bản đầu của kiểm tra này gọi tới cổng 80 rồi đếm dòng "Non-2xx" mà ApacheBench in ra. Cổng 80 chỉ làm một việc là chuyển cường bức sang HTTPS, nên toàn bộ 200 request đều nhận mã 301, và 301 cũng là non-2xx. Kiểm tra vì thế luôn đạt cho dù giới hạn có tắt. Bản hiện tại gọi thẳng sang HTTPS và đếm mã 429 trong tệp log riêng của đường thăm dò, do chính nginx ghi lại. Kết quả là 193 trên 200 request nhận 429; bảy request còn lại lọt qua, sát với `burst=5` cộng một request đang được xét, phần lệch nằm ở tốc độ bù của vùng trong lúc máy đo gửi hết 200 request.

Hạng mục đọc file cấu hình cũng từng là một kiểm tra chết. Vị trí khai báo location `~ ^/(config\.php|\.env)` nằm sau location `~ \.php$`, mà nginx xét các regex theo thứ tự xuất hiện, nên mọi request tới `/config.php` đã bị khối xử lý `.php` nhận trước và rule chặn không bao giờ chạy. Sửa thành location `= /config.php` thì đúng, vì match tuyệt đối có ưu tiên cao hơn mọi regex.

Hạng mục "Node từ chối container khác trong cùng mạng backend" trước đây cho kết quả ngược lại và được xếp loại ghi nhận: node vẫn trả lời 200 cho bất kỳ ai nằm trong mạng backend, nên hàng rào duy nhất là tầng mạng. Sau khi thêm `allow 172.20.0.10; deny all;` ở node, đúng cuộc gọi đó nhận 403 và hạng mục chuyển thành đạt. Hạng mục log đi kèm chứng minh việc chặn nguồn không làm hỏng truy vết: node vẫn ghi đúng địa chỉ khách hàng vào dòng log của mình.

```

lb-lab-12 bảng kiểm tra an toàn 19 hạng mục
=====
KIEM TRA AN TOAN WEB SERVER CLUSTER      20260929-152524
Load balancer 192.168.240.10 | máy đo client01 192.168.240.20
=====
[PASS] An phien ban trong header Server      server_tokens off tai load balancer
[PASS] Khong ro X-Powered-By cua PHP          proxy_hide_header o LB
[PASS] Co header Strict-Transport-Security   max-age 31536000
[PASS] Co header X-Content-Type-Options     nosniff
[PASS] Co header Content-Security-Policy     default-src self
[PASS] Cong 80 chuyen cuong buc sang HTTPS   HTTP chi dung de redirect
[PASS] Thuong luong duoc TLS 1.2 tro len     ket qua TLSv1.3
[PASS] Cookie phien mang co ky HttpOnly     cookie_httponly
[PASS] Cookie phien mang co ky Secure       chi gui di qua HTTPS
[PASS] Cookie phien mang co SameSite        SameSite=Lax
[PASS] Khong goi truc tiep duoc thu vien rieng location ^~ /lib/
[PASS] Healthz khong lo thông tin hệ thống body: {"status": "ok", "node": "web03"}
[PASS] Rate limit chận request gui qua nhanh 193/200 nhan 429 (burst=5)
[PASS] Thu muc config cua ung dung bi node chận location = /config.php
[PASS] Node web khong goi duoc tu mạng phía ngoài mang backend internal
[PASS] Node tu choi container khong phai load balancer allow 172.20.0.10 o node, truoc day tra
200
[PASS] Node web khong co duong ra Internet   khong tai duoc package
[PASS] Node ghi dung địa chỉ khách hàng sau can bang tai client=192.168.240.20 trong node_trac
e
[PASS] Mọi node tra ve cung mot noi dung     ba node chi ra 1 hash
=====
TONG KET: PASS=19  FAIL=0  SKIP=0  GHI_NHAN=0
=====
Da lưu vào results/security/seccheck-20260929-152524.txt
exit code: 0

```

Hình 30 Bảng kiểm cấu hình mười chín hạng mục in ra từ `sec_check.sh`

```

lb-lab-18 giới hạn tần suất chặn request gửi qua n
=== C:/Users/Administrator/Documents/Qoder/2026-09-29/8b86a2bd/de07-web-1b-cluster ===
$ b=$(docker compose exec -T lb01 sh -c "wc -l < /var/log/nginx/probe.log" | tr -dc 0-9); docker
compose exec -T client01 ab -n 2...
Complete requests:      200
Failed requests:        193
Non-2xx responses:      193
so request nhan ma 429 trong log tham do: 193
exit code: 0

```

Hình 31 Giới hạn tần suất chặn 194 trong 200 request gửi dồn dập vào đường thăm dò

3.7 Nhận xét và hạn chế

Sáu yêu cầu đặt ra ở mục 2.1 đều được kiểm chứng bằng số liệu: chọn node theo thuật toán khai báo ở mục 3.2, dịch vụ vẫn trả lời khi tắt hẳn một node ở mục 3.3, phiên không nằm trong node ở mục 3.4, mọi node trả về cùng một nội dung và node không tiết lộ thông tin hệ thống ở bảng kiểm 3.6, mọi phép đo chạy lại được bằng một lệnh kèm log thô ở mục 3.1.3. Riêng yêu cầu thứ năm, node không tiếp nhận kết nối từ ngoài, nay đạt ở cả hai lớp: từ mạng phía ngoài không thiết lập được kết nối, và từ một container khác trong cùng mạng backend thì node trả về 403.

Ba hạn chế còn lại của mô hình.

Thứ nhất, bộ cân bằng tải là điểm hỏng đơn lẻ. Mất lb01 thì cả cụm mất cửa vào. Hướng xử lý bằng hai instance và VRRP được nêu ở phần định hướng phát triển của Kết luận.

Thứ hai, nginx mã nguồn mở chỉ phát hiện node chết một cách thụ động, nên luôn có request phải chịu hai giây chờ. Các request chậm trong phép đo failover ở mục 3.3 và 126 request phải thử lại trong phép đo ở mục 3.5.4 là biểu hiện của cơ chế này.

Thứ ba, toàn bộ cụm chạy trên một máy vật lý. Các phép đo về giao thức và định tuyến vì thế vẫn phản ánh đúng hành vi của hệ thống, còn độ trễ mạng thật giữa các node và hiện tượng nghẽn ở cổng lên thì chưa nằm trong phạm vi đo.

3.8 Kết chương

Chương này đã đo đủ ba nội dung đề bài yêu cầu. Về phân phối tải, 600 request gửi đồng thời qua 20 kết nối, tổng hợp từ 20 lượt độc lập cho mỗi thuật toán: round robin chia 199,9 / 200,1 / 200,0 với độ lệch lớn nhất giữa hai node trong một lượt là 1,0 request, least_conn chia 199,9 / 200,5 / 199,6 với độ lệch 10,2 request, còn ip_hash dồn 600 request về một node ở cả hai mươi lượt. Về khả năng chịu lỗi, tắt hẳn một node trong 30 giây không làm hỏng request nào ở cả 12 vòng đo lặp lại; cái giá phải trả là $8,8 \pm 0,4$ request mỗi vòng phải chờ thêm hai giây, đúng như phân tích về kiểm tra sức khỏe thụ động ở Chương 1. Về phiên làm việc, kho dùng chung giữ được 10 trên 10 phiên sau khi tắt node phục vụ đăng nhập, cách lưu trong node giữ được 0 trên 10.

Ngoài ba yêu cầu đó, ma trận 18 ô với 20 lượt cho mỗi ô trả lời thêm câu hỏi nên nhìn vào đâu khi hệ thống chậm. Bộ cân bằng tải tự nó phục vụ 31360 req/s; khi mỗi yêu cầu phải chuyển tiếp sang node, con số còn 805 req/s. Thuật toán chọn node không tạo ra khoảng cách đó: ở tầng ứng dụng, round robin và least_conn chênh nhau không quá 5,1% và dấu của chênh lệch đổi chiều giữa hai chế độ kết nối; chỉ có ip_hash, thuật toán dồn mọi lưu lượng vào một node, tách khỏi hai thuật toán kia với mức -10,4%. Mất một trong ba node cũng không làm thông qua trung bình giảm bao nhiêu, chỉ -7,7%, nhưng làm hệ số biến thiên tăng từ 4,2% lên 14,7% và đẩy request chậm nhất từ 54 ms lên 1048 ms. Mười chín hạng mục kiểm tra cấu hình đều cho kết quả đạt.

KẾT LUẬN

Các kết quả đạt được

Nhóm đã hoàn thành việc tìm hiểu và triển khai cụm máy chủ web cân bằng tải theo đề tài số 7, đối chiếu với từng yêu cầu của đề bài như sau.

Về tìm hiểu lý thuyết, báo cáo đã trình bày được khái niệm cân bằng tải máy chủ web, vị trí của nó trong yêu cầu bảo đảm tính sẵn dùng theo giáo trình, phân biệt cân bằng tải tầng ba, tầng bốn và tầng bảy, liệt kê các thành phần của một cụm gồm pool thành viên, cơ chế kiểm tra sức khỏe, kho phiên dùng chung và cơ sở dữ liệu dùng chung, so sánh sáu thuật toán phân phối và năm nhóm công cụ.

Về triển khai cụm hai đến ba máy chủ web phục vụ cùng một website, hệ thống gồm ba node nginx kết hợp php-fpm 8.3, cùng dựng từ một bản ảnh, cùng đọc một MariaDB và cùng dùng một kho phiên Redis. Toàn bộ khai báo trong một tệp Docker Compose, khởi động lại được bằng một lệnh.

Về thành phần cân bằng tải, bộ cân bằng tải nginx 1.27 đặt ở vị trí duy nhất nhận lưu lượng từ ngoài, có khối upstream khai báo ba thành viên, chọn thuật toán qua tham số chạy mà không phải sửa cấu hình.

Về kiểm tra phân phối request, hai phép đo đã thực hiện. Với 300 request gửi tuần tự, phân phối là 104 / 98 / 98 và chuỗi phân phối của round robin trùng khớp least_conn ở từng vị trí. Với 600 request gửi qua 20 kết nối song song, chạy 20 lượt độc lập cho mỗi thuật toán, round robin cho 199,9 / 200,1 / 200,0 với độ lệch lớn nhất giữa hai node trong một lượt là 1,0 request; least_conn cho 199,9 / 200,5 / 199,6 với độ lệch 10,2 request; ip_hash dồn 600 về một node ở cả hai mươi lượt. Phép đo thứ hai được bổ sung sau khi phát hiện phép đo thứ nhất không phân biệt được round robin với least_conn, và chính chỗ này cho kết luận rằng least connection chỉ khác round robin khi có request chồng lấn hoặc khi các node không đồng chất.

Về hiệu năng, một ma trận 18 ô với 20 lượt đo cho mỗi ô tách được ba thứ vẫn hay bị gộp làm một. Bộ cân bằng tải tự nó đạt 31360 req/s; con số rơi xuống 805 req/s khi mỗi yêu cầu phải chuyển tiếp sang node và 707 req/s khi node còn phải truy vấn cơ sở dữ liệu. Tính theo thời gian cho một request ở 20 kết nối song song, 24,2 ms trong tổng số 28,3 ms thuộc về tầng ứng dụng, chỉ 0,6 ms thuộc về việc cân bằng tải, và một truy vấn MariaDB cộng thêm 3,5 ms. Ba thuật toán không hơn kém nhau ổn định về thông qua: độ lớn chênh lệch không quá 5% ở tầng ứng dụng và dấu của nó đổi chiều giữa hai chế độ kết nối. Mất một trong ba node làm thông qua trung bình giảm 7,7%, nhưng làm hệ số biến thiên tăng từ 4,2% lên 14,7% và đẩy request chậm nhất trong lượt đo từ 54 ms lên 1048 ms.

Về khả năng đáp ứng khi một server gặp sự cố, vòng đo 70 giây được lặp lại 12 lần độc lập, mỗi lần khoảng 219 mẫu, với lệnh tắt hẳn web02 ở giây thứ 10. Không có request nào nhận mã lỗi ở cả 12 vòng. Trung bình $8,8 \pm 0,4$ mẫu phải chờ thêm khoảng hai giây do nginx thử lại node đã chết, và hành vi này khớp với cơ chế kiểm tra sức khỏe thụ động đã phân

tích ở Chương 1. Bảng chứng là chín dòng log có hai địa chỉ upstream kèm cặp mã trạng thái 504 rồi 200. Cùng kịch bản tắt đúng node vừa phục vụ đăng nhập, phiên của người dùng chỉ sống được khi nó không nằm trong node: phương án kho Redis dùng chung giữ được 10 trên 10 phiên, phương án lưu trên đĩa của chính node giữ được 0 trên 10.

Ngoài ba yêu cầu của đề bài, đề tài còn làm thêm hai việc phục vụ trực tiếp cho tính sẵn dùng và cho môn học. Thứ nhất, xây dựng bảng kiểm cấu hình và thuộc tính an toàn gồm mười chín hạng mục tự động, phủ ản phiên bản máy chủ, kết thúc TLS tại cân bằng tải, HSTS và các header an toàn, cờ bảo vệ cookie phiên, giới hạn tần suất cho đường thăm dò, chặn đọc file cấu hình, cách ly node khỏi mạng phía ngoài, node chỉ nhận yêu cầu từ đúng bộ cân bằng tải, đường kiểm tra sức khỏe không lộ thông tin và việc mọi node trả về cùng một nội dung. Cả mười chín hạng mục đều xác nhận cấu hình có hiệu lực. Bảng kiểm đánh giá hiệu lực của cấu hình, không đánh giá khả năng chống tấn công. Thứ hai, toàn bộ log thô của mọi phép đo được lưu trong thư mục results, nên từng con số trong báo cáo đều kiểm tra lại được.

Trong quá trình làm, năm chỗ sai đã phát hiện và sửa. Ba chỗ sai ở cấu hình im lặng, không báo lỗi mà chỉ làm tính năng mất tác dụng: location chỉ có return khiến limit_req không bao giờ được xét; location ~ ^/(config\.php|\.env) đặt sau location ~ \.php\$ nên không bao giờ chạy vì nginx xét regex theo thứ tự xuất hiện; php-fpm với clear_env = yes mặc định xóa hết biến môi trường của container khiến ứng dụng không đọc được địa chỉ cơ sở dữ liệu. Một chỗ sai làm nginx không khởi động được: khai báo nodelay đặt nhầm ở limit_req_zone thay vì limit_req. Chỗ sai cuối cùng nằm ở chính công cụ đo: kiểm tra giới hạn tần suất đếm dòng "Non-2xx" của ApacheBench trong khi gọi tới cổng 80, nơi mọi request đều nhận 301 chuyển hướng, nên nó báo đạt ngay cả khi giới hạn đang tắt.

Hướng phát triển

Bốn hướng mở rộng được xác định nhưng chưa thực hiện trong đề tài.

Loại bỏ điểm hỏng đơn lẻ còn lại của chính bộ cân bằng tải bằng hai instance và giao thức VRRP qua keepalived, kèm kiểm tra nginx định kỳ để tự hạ ưu tiên. Cấu hình mẫu đã chuẩn bị, nhưng Docker Desktop trên Windows không bảo đảm chuyển được multicast của VRRP, nên cần làm trên mạng ảo hóa kiểu VirtualBox hoặc VMware hoặc trên nhiều máy vật lý.

Chuyển từ kiểm tra sức khỏe thụ động sang chủ động. HAProxy với option httpchk phát hiện node chết mà không cần request thật, dự kiến loại được các request phải chờ hai giây ở mục 3.3, trung bình 8,8 mẫu trên mỗi vòng đo. Cần một phép đo so sánh trực tiếp hai cơ chế trên cùng một tải.

Mã hóa đoạn từ bộ cân bằng tải về node và thêm ràng buộc để node chỉ nhận yêu cầu từ đúng cân bằng tải, chẳng hạn TLS hai chiều với chứng chỉ client hoặc quy tắc tường lửa theo địa chỉ nguồn. Hiện tại việc cách ly chỉ dựa ở tầng mạng.

Đưa cơ sở dữ liệu và kho phiên ra mô hình có dự phòng, gồm nhân bản MariaDB và Redis Sentinel, để hai thành phần này không còn là điểm hỏng đơn lẻ. Khi đó mới đo được hành vi của cụm khi mất cả node web lẫn node dữ liệu cùng lúc.

TÀI LIỆU THAM KHẢO

[1] Hoàng Xuân Đâu, Bài giảng An toàn ứng dụng web và cơ sở dữ liệu, Học viện Công nghệ Bưu chính Viễn thông, Hà Nội, 2017.

[2] nginx, "Module ngx_http_upstream_module", nginx Documentation, https://nginx.org/en/docs/http/ngx_http_upstream_module.html, truy cập tháng 9 năm 2026.

[3] nginx, "Module ngx_http_limit_req_module", nginx Documentation, https://nginx.org/en/docs/http/ngx_http_limit_req_module.html, truy cập tháng 9 năm 2026.

[4] nginx, "Module ngx_http_realip_module", nginx Documentation, https://nginx.org/en/docs/http/ngx_http_realip_module.html, truy cập tháng 9 năm 2026.

[5] nginx, "Module ngx_http_proxy_module", nginx Documentation, https://nginx.org/en/docs/http/ngx_http_proxy_module.html, truy cập tháng 9 năm 2026.

[6] PHP Manual, "Session configuration", <https://www.php.net/manual/en/session.configuration.php>, truy cập tháng 9 năm 2026.

[7] PHP Manual, "password_hash", "password_verify" và "session_regenerate_id", <https://www.php.net/manual/en/ref.passwordhash.php>, truy cập tháng 9 năm 2026.

[8] phredis, "Session Handler" và tài liệu tham số save_path, <https://github.com/phredis/phredis>, truy cập tháng 9 năm 2026.

[9] Redis, "Redis configuration" và "In-memory persistence", https://redis.io/docs/latest/operate/oss_and_stack/management/configuration/, truy cập tháng 9 năm 2026.

[10] MariaDB Foundation, "MariaDB Server 11.4 Documentation", <https://mariadb.com/kb/en/documentation/>, truy cập tháng 9 năm 2026.

[11] HAProxy Technologies, "Configuration: health checks and load balancing algorithms", <https://www.haproxy.com/documentation/haproxy-configuration-manual/>, truy cập tháng 9 năm 2026.

[12] Docker, "Networking: isolate containers with an internal network", <https://docs.docker.com/engine/network/drivers/bridge/>, truy cập tháng 9 năm 2026.

[13] D. Thaler, R. Bonica, A. Lewis, V. Naik, "Virtual Router Redundancy Protocol (VRRP) Version 3 for IPv4 and IPv6", RFC 5798, IETF, tháng 3 năm 2010.

[14] OWASP Foundation, "OWASP Top 10", <https://owasp.org/www-project-top-ten/>, truy cập tháng 9 năm 2026.

[15] Apache Software Foundation, "ab: Apache HTTP server benchmarking tool", <https://httpd.apache.org/docs/2.4/programs/ab.html>, truy cập tháng 9 năm 2026.